CAN/LIN Protocol

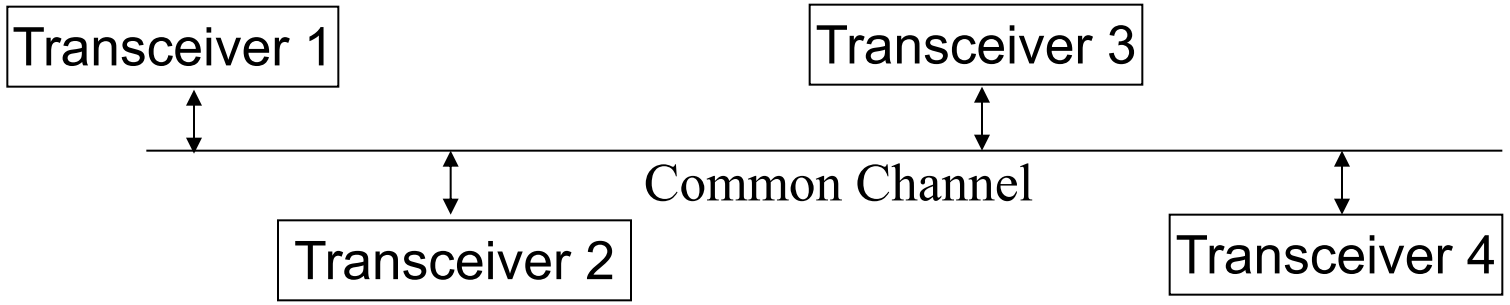
Subra Ganesan
Oakland University

Agenda

- Vehicle Multiplexing – Introduction
- CAN Systems
  - CAN System - summary
  - Messages, Frame Structure
  - Chrysler CAN VMM
  - Error handling
  - Physical Layer
  - CAN Tools
- LIN Systems
  - LIN System – summary
  - Physical Layer
  - Messages, Frame Structure
  - Schedule Tables
  - LIN Tools
- Chrysler Networking Process

What are CAN & LIN?

- CAN and LIN are in-vehicle **multiplexing** Technologies
  - **Multiplexing**: one common channel is used for data transmission and reception between multiple stations

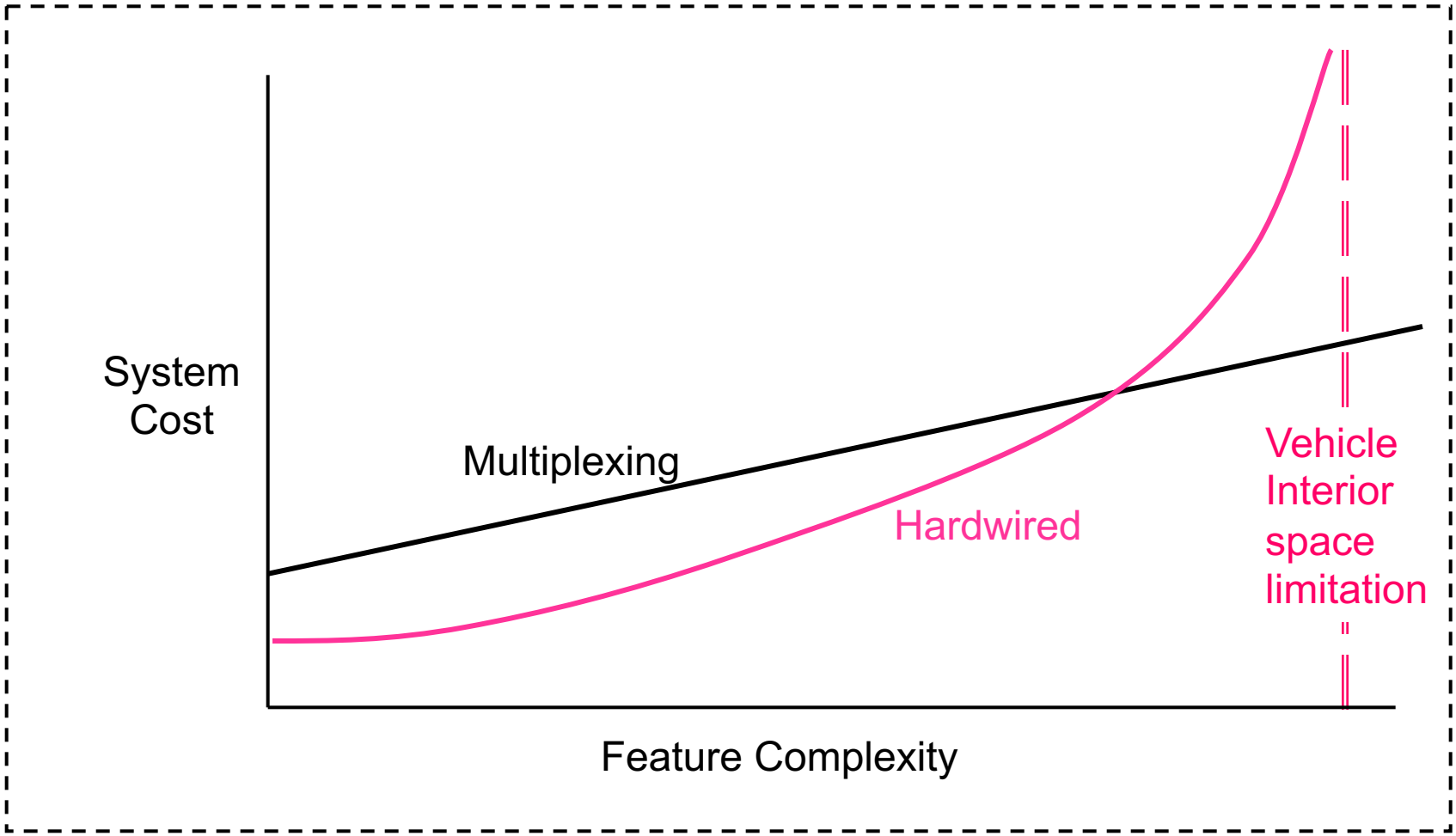


Why Vehicle multiplexing?

Requirements on safety, emissions, fuel economy and competitive new features

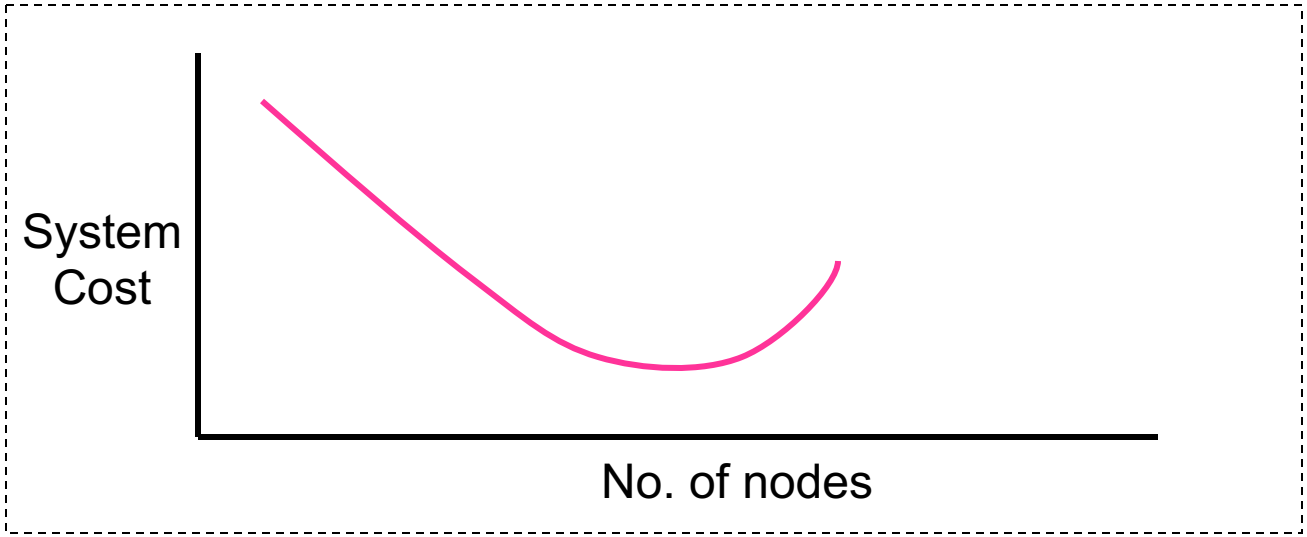
- Size and weight reduction
- Avoid complex wiring and redundant sensors
- Diminishing installation space
- Modularity
- Improved diagnosis and serviceability
- Improve cost

System Cost


System Cost
Multiplexing
Hardwired
Vehicle Interior space limitation
Feature Complexity

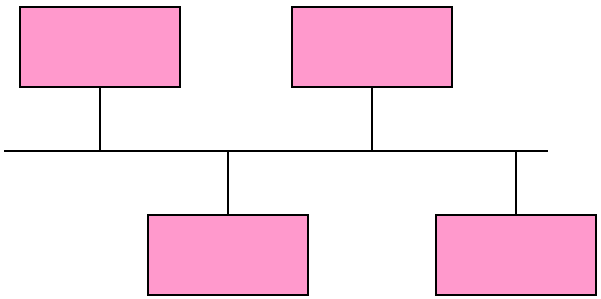
Vehicle Multiplexing Options

- No multiplexing
- Put all sensors and actuators on the bus
- Have a handful of multiplexing nodes (ECUs)

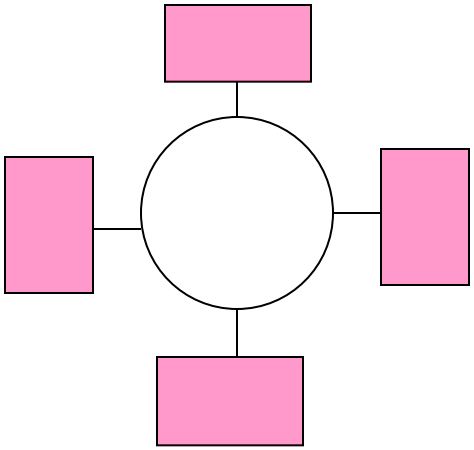


Local network

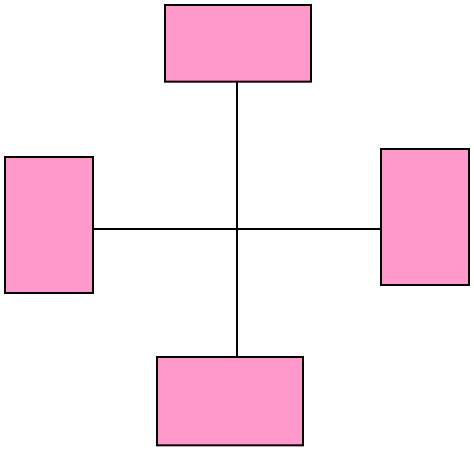
- Confined in a small area
- All stations are attached to a common wire
- Transmission from one node propagates in both directions and can be received by all other nodes



Bus local network



Ring local network



Star local network

Multiplexing at Chrysler

Bus	Speed	Header	Physical Medium
CCD	7.8125 KBS	8-bit	Twisted wire
J1850	10.4 KBS	8-bit	Single wire
CAN	$\leq$ 1 MBPS	11-bit & 29-bit	Twisted Pair
LIN	$\leq$ 19.2 KBS	8-bit	Single wire

Vehicle Multiplexing

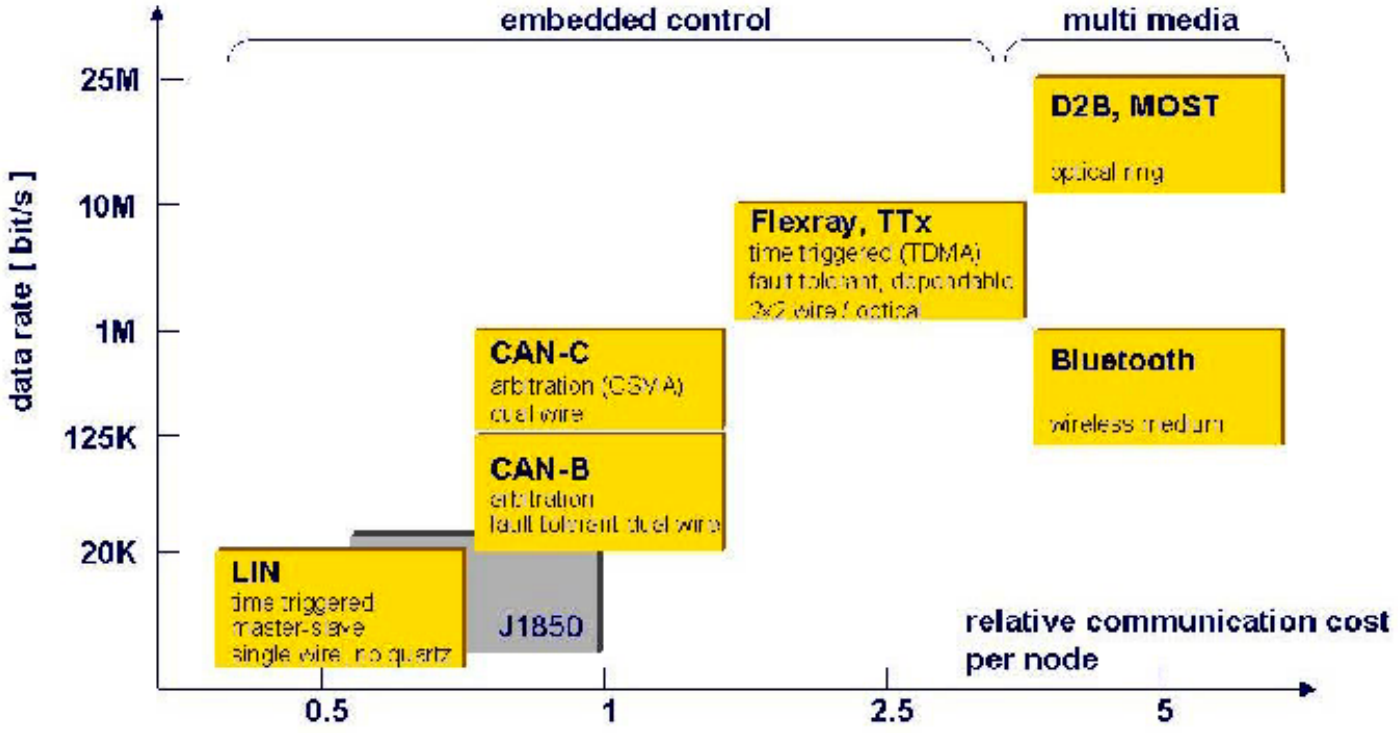
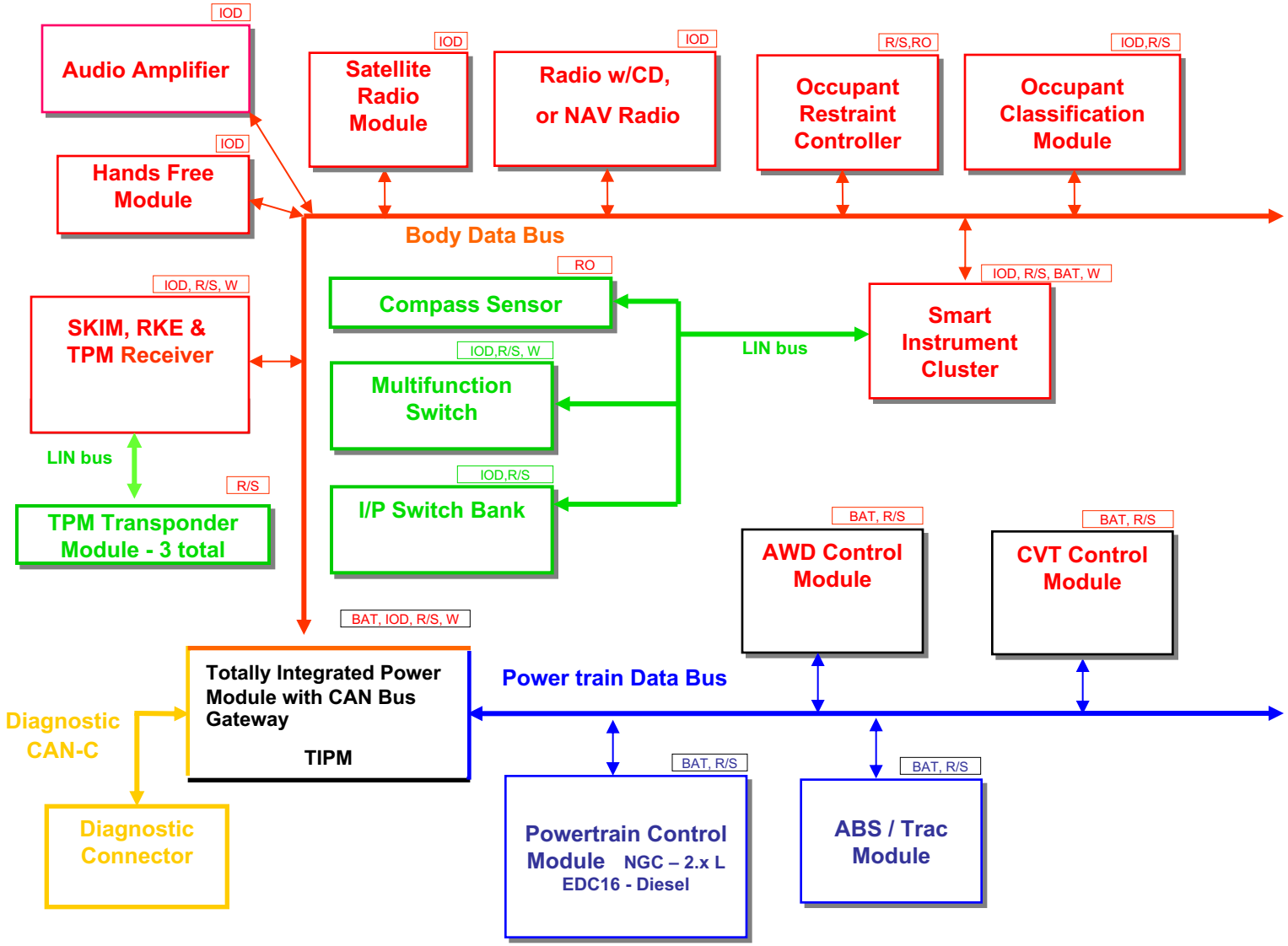


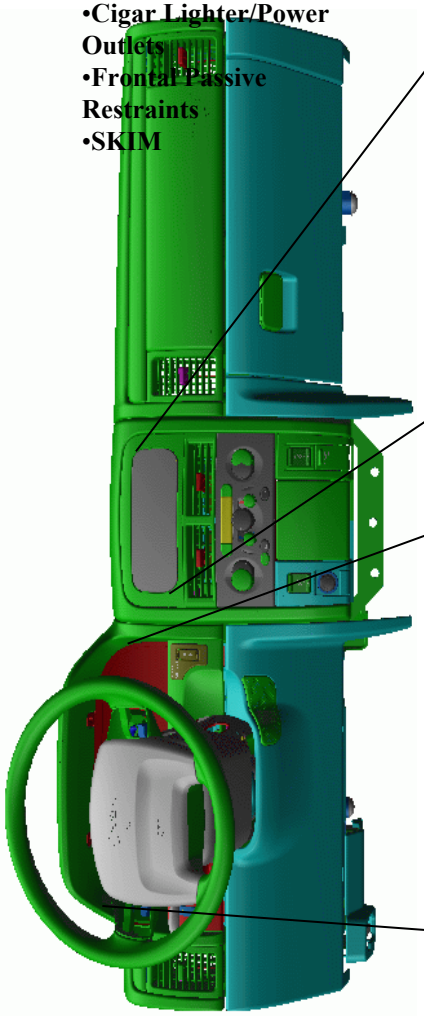
Figure 1: Major Network Protocols in Vehicles

Typical CAN Network



Future Car network

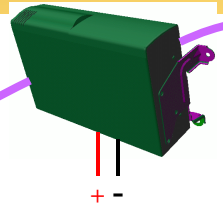
INSTRUMENT PANEL
- **Instrument Cluster**
- **Junction Block Module**
- **HVAC Unit/Controller**
- **Radio/ICU**
- **Speakers**
- **Switches**
- **Lamps**
- **Cigar Lighter/Power Outlets**
- **Frontal Passive Restraints**
- **SKIM**



RADIO/ICU
- **CAN B / MOST Gateway**
- **NAV**
- **HVAC**



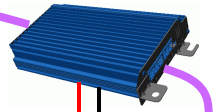
CD CHANGER



DVD PLAYER



DISTRIBUTED AMPLIFIERS

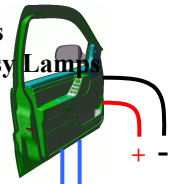


NAV

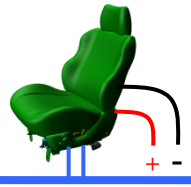
PHONE

MOST
Media Oriented Systems Transport

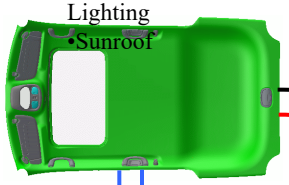
DOORS
- **Window Motors**
- **Lock Motors**
- **Switches (window, lock, mirror, memory, disarm, ajar)**
- **Mirrors**
- **Courtesy Lamps**



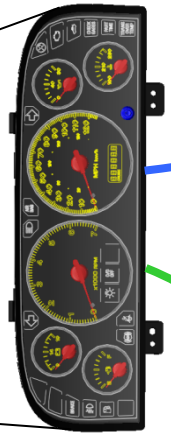
SEATS
- power
- heat
- memory
- switches



OVERHEAD
- EVIC
- Courtesy/Dome Lighting
- Sunroof

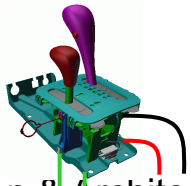


INSTRUMENT CLUSTER
- CAN B / CAN C Gateway

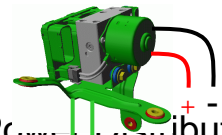


CAN B
Low Speed Data Bus

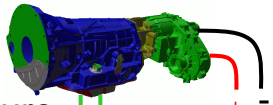
SHIFTER
- Shift by Wire
- AutoStick
- Shift Interlock
- Illumination



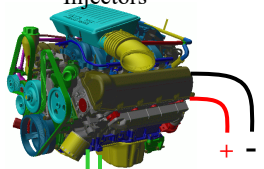
ABS



TRANSMISSION
- **Transmission Controller**
- **Transfer Case Controller**



ENGINE
- **Engine Controller**
- Sensors
- Injectors



CAN C
High Speed Data Bus

CAN System: Summary

- Multi-master priority-based bus access
- Non-destructive priority-based arbitration
- Multicast frame transfer by acceptance filtering
- Configuration flexibility
- system-wide data consistency

CAN System: Summary

- Error detection and error signalling
- Automatic retransmission of frames that have lost arbitration or have been destroyed by errors during transmission
- Distinction between temporary errors and permanent failures of nodes and autonomous switching-off of defective nodes
- Bit rate: 5 Kbit/s – 1 Mbit/s

Message Types

Messages are one of four different types, called frames

- **Data Frame** : used to transmit up to 8 bytes of data
- **Remote Transmission Request Frame** : used to request a Data Frame (presently not used at DC)
- **Error Frame** : used to indicate a bus error
- **Overload Frame** : used to create an extra delay between Data Frames or Remote Frames (presently not used at DC)

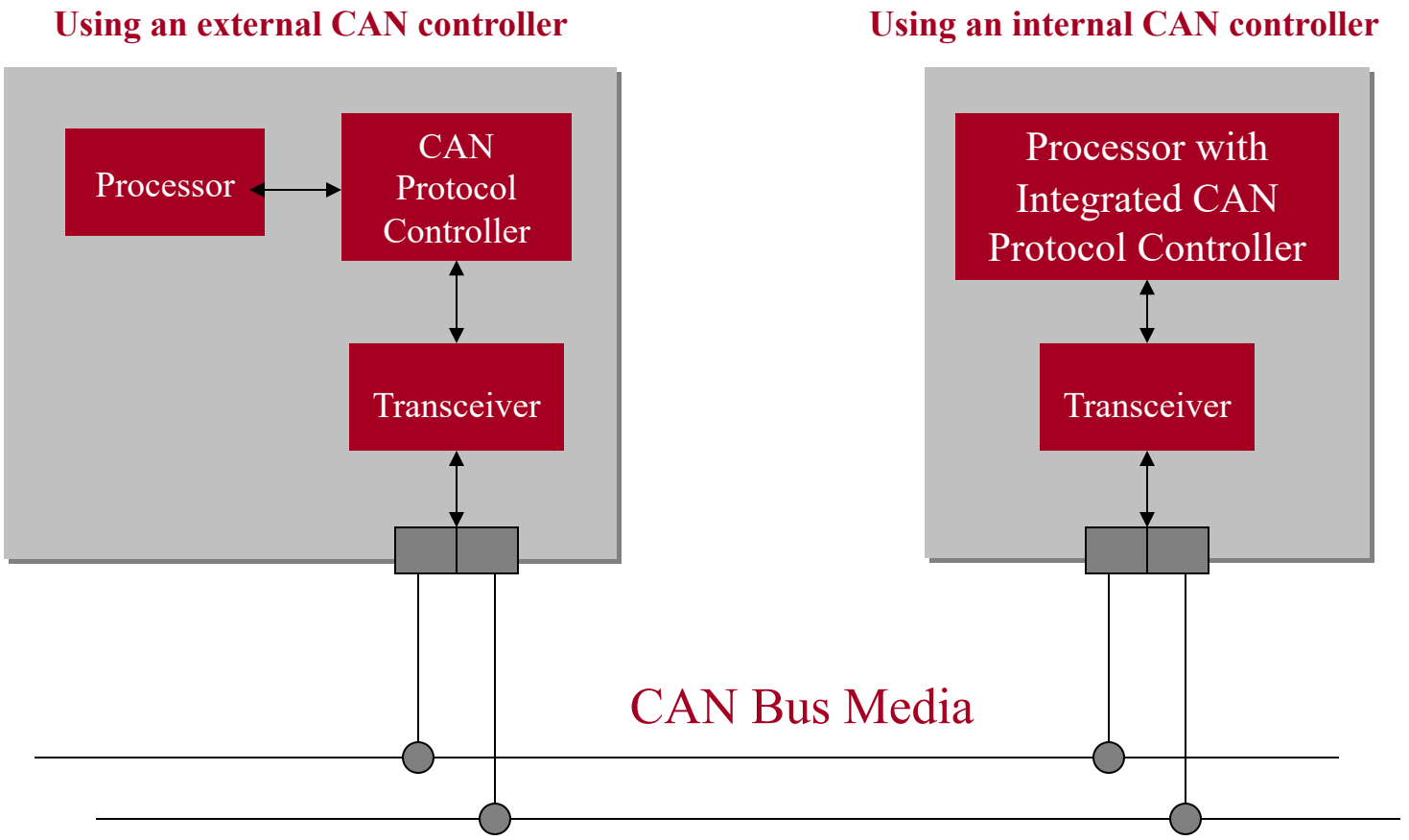
Ethernet and CAN

- By comparison, Ethernet's familiar Collision Sensing Multiple Access (CSMA) protocol allows all nodes equal access to the bus to transmit data. It resolves conflicts by having the nodes back off and rebroadcast after a pseudo-random time delay. The scheme works, but is also inefficient.
- During high load conditions, for example, nodes trying to broadcast spend a good deal of their time getting on the bus, backing off, waiting to get on again, and colliding again. As a result, only about 60% of the theoretical bus throughput is ever utilized and Ethernet must go to higher and higher bus speeds to increase throughput.
- CAN's protocol, on the other hand, provides nodes

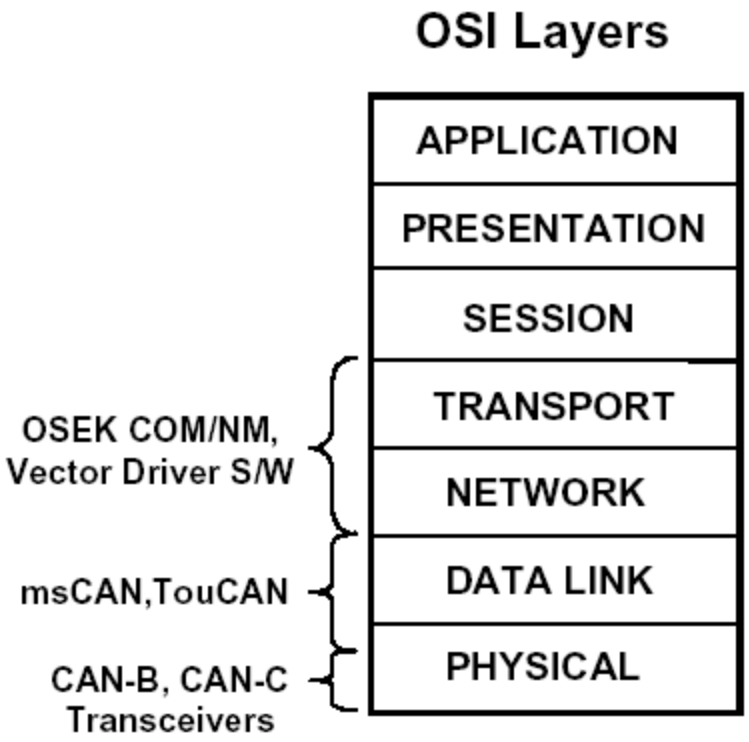
Non-destructive Arbitration

- The net result of non-destructive arbitration is bandwidth utilization of 80% to 90% and very high reliability.
- The scheme of having each node monitor its own broadcast and that of other nodes limits its maximum throughput to 1 Mbit/sec at the bus length of 30 meters.
- Maximum bit rate can be increased by shortening the bus length.

Hardware Implementations



ISO/OSI Structure of a CAN Node



CAN Protocol

- **DATA LINK LAYER**
  - LOGICAL LINK CONTROL (LLC) SUB-LAYER
    - Acceptance Filtering
    - Overload Notification
    - Recovery Management
  - MEDIUM ACCESS CONTROL (MAC) SUB-LAYER
    - Data Encapsulation/Decapsulation
    - Frame Coding (Bit Stuffing/Unstuffing)
    - Medium Access Management
    - Error Detection/Signaling
    - Acknowledgement
    - Serialization/Deserialization
- **PHYSICAL LAYER**
  - Bit Encoding/Decoding
  - Bit Timing
  - Synchronization

J1850 (continued)

- Time division multiplexing
- Low radiated emission- minimized bus transitions per data bit
- VPW – able to compensate for clock mismatch and ground offset
- 64 micro seconds – dominant ONE bit
- 128 microseconds- passive ZERO bit
- High--- 4.25 volts to 20 V
- Low- 3.5 volts to 0 V

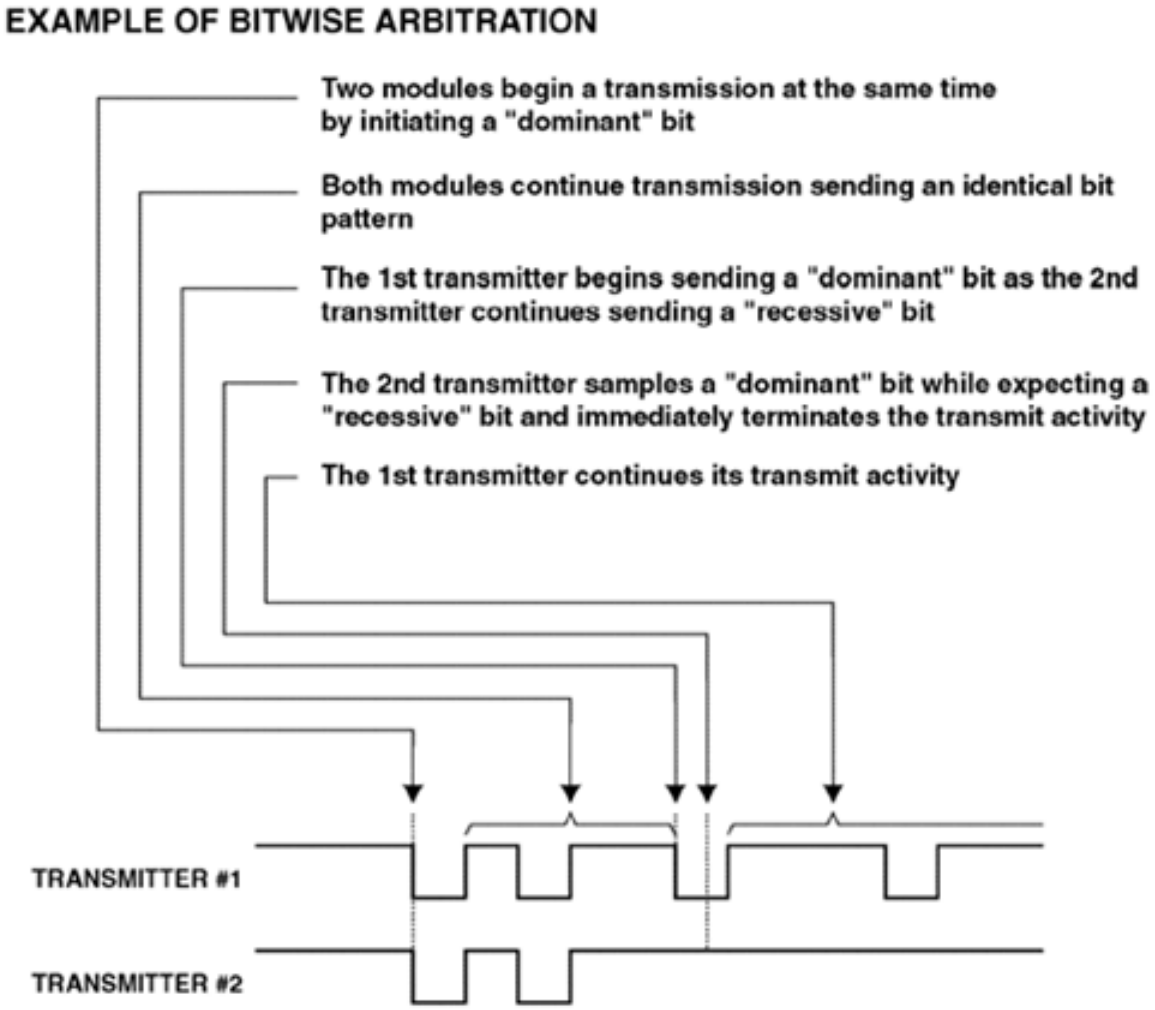
Message Transfer

- **Any one of free node can start to transmit.**
- **Bus conflict resolved by bit-wise arbitration.**
- **The highest priority message wins bus access.**
- **Transmitting nodes which loose arbitration become receivers.**
- **No data or time is wasted, someone always wins.**
- **Nodes which loose arbitration will automatically retransmit.**
- **Arbitration Field (identifier) determines the message priority.**

Message Transfer (contd…)

- **All nodes check the consistency of messages received and will flag an inconsistent message to the entire network.**
- **All receiving nodes will acknowledge a valid message.**
- **A message is received correctly by all nodes or no nodes.**
- **All nodes apply Message Filtering to decide whether to accept a message.**
- **Any number of nodes can simultaneously**

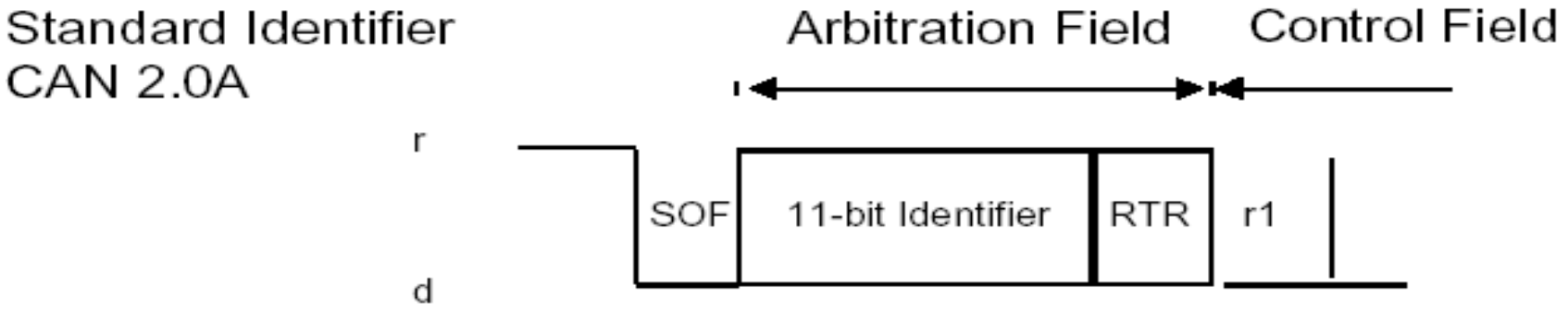
Bitwise Arbitration

- 

Bitwise Arbitration Rules

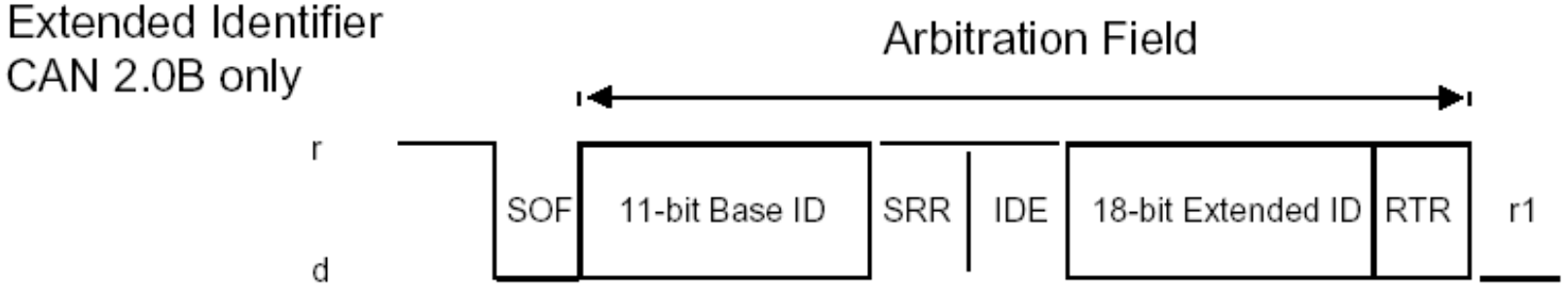
- A "dominant" bit always wins over a "recessive" bit (similar to open collector logic)
- A transmitter always compares its transmit bit with its receive bit.
- If a transmitter detects that the receive bit is not the same as the transmit bit, then the transmitter will immediately stop its transmission. (With some dependencies, a subsequent re-transmission may occur)

CAN Arbitration Field 2.0A



- The 11-bit Identifier is labeled ID10 to ID0 in CAN2.0A.
- The Identifier is transmitted most significant bit first.
- The 7 most significant bits of the Identifier must not all be recessive, per the CAN Specification.
- RTR bit is dominant for a Data Frame, recessive for a Remote Frame. r1 is always dominant.

CAN Arbitration Field 2.0B (contd..)



- The 11-bit Base Identifier is labeled ID28 to ID18.
- The 18-bit Extended Identifier is labeled ID17 to ID0.
- The Identifier is transmitted most significant bit first.
- The 7 most significant bits of the Identifier must not all be recessive.
- SRR bit is recessive for an Extended Identifier
- IDE bit is recessive for an Extended Identifier.
- RTR bit is dominant for a Data Frame, recessive for a Remote Frame

CAN Protocol

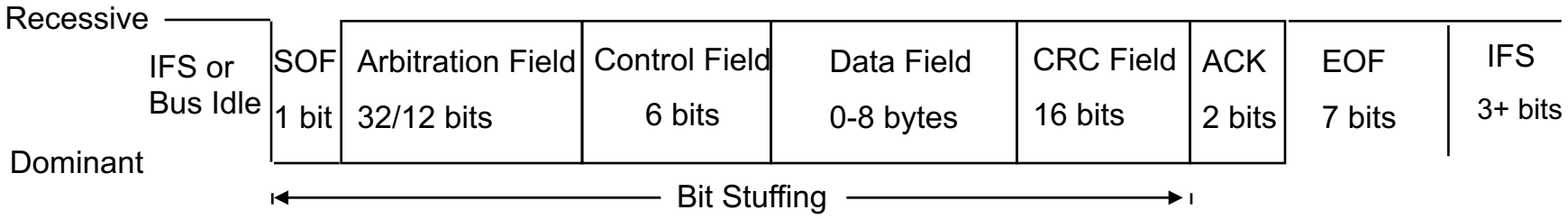
- CAN is a serial communication protocol
- The bit-stream is coded as non-return-to zero (NRZ) method



– **Bit Stuffing is used**: whenever 5 consecutive bits of equal value are transmitted, an extra bit of complementary value is inserted into the bit stream

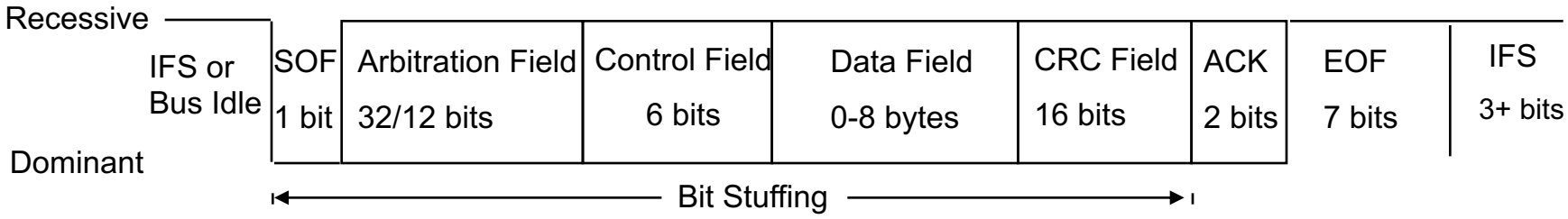
– **CRC** (Cyclic Redundancy Check): An error detection scheme based on n-bit division

CAN Data Frame



- Inter-Frame Space (IFS): A node may start transmitting a Data Frame.
- Start Of Frame (SOF): Receivers synchronize at the leading edge.
- Arbitration Field: Arbitration based on message ID
  - 12 (11+1) bits for a Standard Data Frame (CAN2.0A/B)
  - 32 (29+3) bits for a Extended Data Frame (CAN2.0B only).

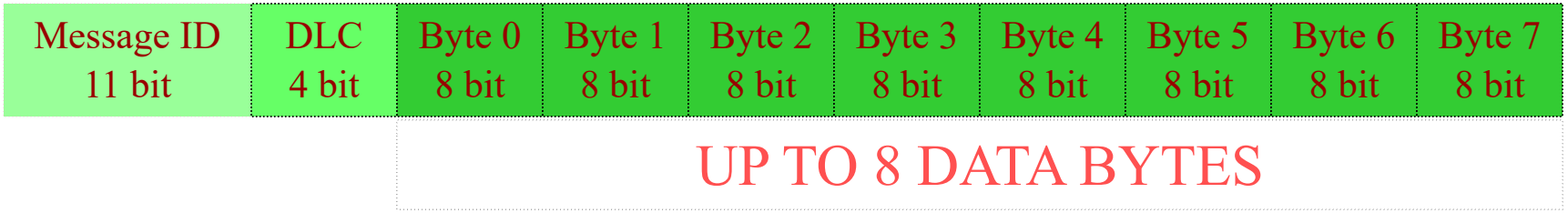
CAN Data Frame



- Control Field: Defines Data/RTR frame & Data length
- Data Field: 1-8 bytes of data
- Cyclic Redundancy Check Field (CRC): 15-bit CRC + delimiter
- Acknowledgement Field (ACK): 1-bit Acknowledgment + delimiter
- End Of Frame Field (EOF): 7/8 recessive bits

CAN Data Frame (Contd.)

- A simplified Data Frame Structure



Chrysler CAN VMM
(Vehicle Message Matrix)

VMM describes the message contents with ID, message structure, and signal description

It is released in two formats

- EXCEL Format
  - Allows user-friendly browsing and printout
- DBC Format
  - Industry standard format for CAN messages that is used for code generation and driving a number of CAN tools





Chrysler Networking Process

CAN Mailbox – repository for CAN information required by suppliers:

- VMMs
- CANoe Models
- Others.

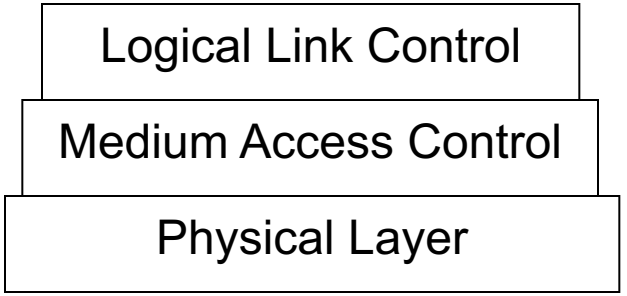
Supplier access granted through Covisint supplier portal – info available on CAN Mailbox

VMM Development

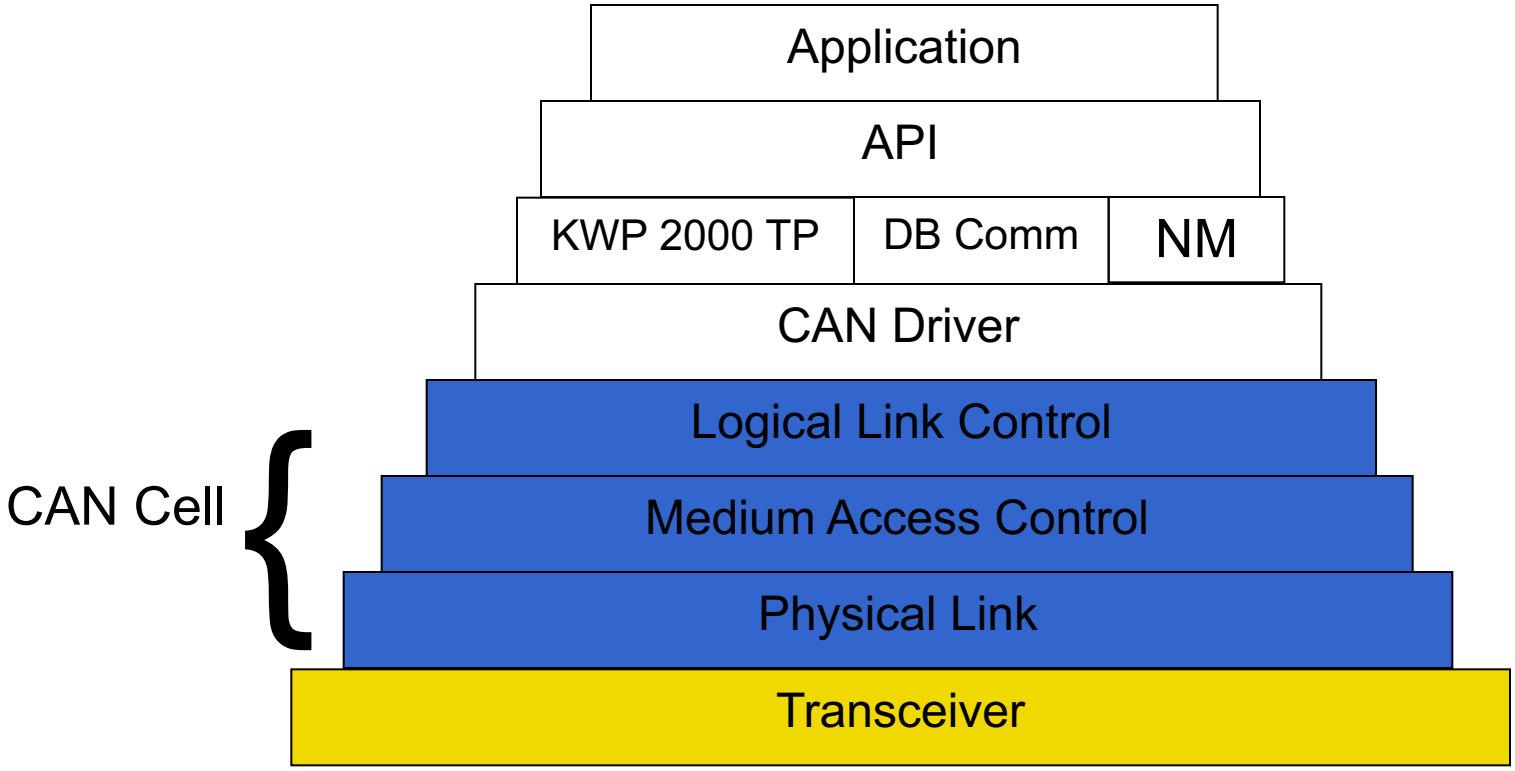
- VMMs are release 4 times/Year, independent of vehicle development timing.
- Timing available on CAN Mailbox
- Vehicle Integration Teams determine which VMM each ECU should use.
- Intranet based Change Request Process

CAN Cell structure

- **LOGICAL LINK CONTROL (LLC)**
  - **Provide service access points**
  - **Acceptance Filtering**
  - **Overload notification**
- **MEDIUM ACCESS CONTROL (MAC)**
  - **Manage access control**
  - **Data En(De)capsulation**
  - **Bit (un)stuffing**
  - **Error Detection/signaling**
  - **Message Acknowledgement**
- **PHYSICAL LINK**
  - **Bit En(De)coding**
  - **Bit Timing**
  - **Clock synchronization**



Standard Software



CAN Error detection

The following measures are provided:

- **Bit Error**: Transmitter detected different bit level compared to the sent value
- CRC Error: At receiver, calculated CRC does not match
- Stuff Error: At receiver, six consecutive similar bits are observed
- Form Error: Frame format is violated
- Acknowledge Error: Transmitter failed to observe an acknowledgement

CAN Error Detection

- Bit Error : a Bit Error is detected by a transmitter if the bit value received is different from the bit value transmitted.
- Exceptions:- sending a recessive bit and receiving a dominant bit during the Arbitration Field or the Ack Slot, or during a Passive Error flag.
- Stuff Error : a Stuff Error is detected by a receiver if 6 consecutive bit values are received during a message field that should be encoded by bit stuffing.
- CRC Error : a CRC Error is detected by a receiver if the CRC calculated by the receiver is different from the CRC received in the CRC Sequence field.
- Form Error : a Form Error is detected by a receiver if

CAN Error Signaling

- A node which detects an error transmits an Error Flag.
- When a Bit Error, a Stuff Error, a Form Error or an Acknowledge Error is detected, the Error Flag is started at the next bit time.
- When a CRC Error is detected, the Error Flag is started at the bit after the ACK Delimiter (unless an Error Flag for another error condition has already started).
- The transmission of a message in which an error is detected is recognized as not complete. The transmitting node will arbitrate for bus access after the Interframe Space following the Error Frame in an

CAN Active Error Frame

- An Error Frame consists of an Error Flag followed by an Error Delimiter.
- There are 2 types of Error Flag: Active Error Flag and Passive Error Flag.
- An Active Error Flag is transmitted when an error is detected by an Error Active node.
- An Active Error Flag consists of 6 dominant bits, which overwrite other bits on the bus and break the rule of Bit Stuffing or destroy the ACK field or EOF field.
- As a consequence, all other nodes detect an error, and transmit their own Error Flag, (and so no nodes receive the original message).
- The number of dominant bits monitored on the bus

CAN Fault Confinement

- Fault Confinement attempts to identify permanent failures.
- Nodes that persistently generate errors are prevented from disturbing communication between other nodes on the bus.
- Each CAN node maintains a **Transmit Error Counter** and a **Receive Error Counter**
- Each time an error is detected the error counters go up while after a successful transmission/reception it goes down
- A node may be (error) normal, error active, error passive or Busoff

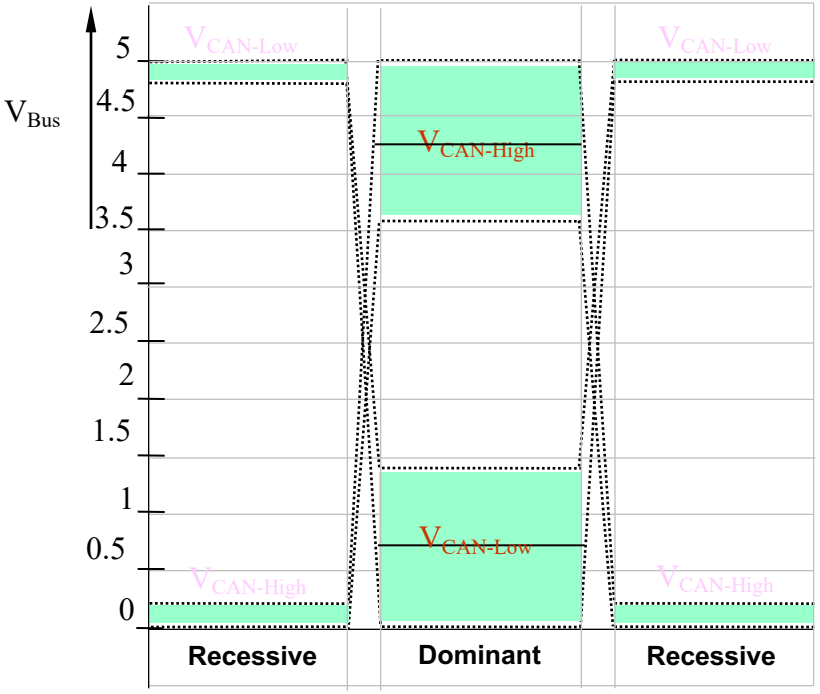
Clock Synchronization

- Receiving nodes on the bus must synchronize their clocks
  - Hard synchronizations are done at the beginning of each message
  - Re-synchronization is done at each bit level by lengthening or shortening the bit time

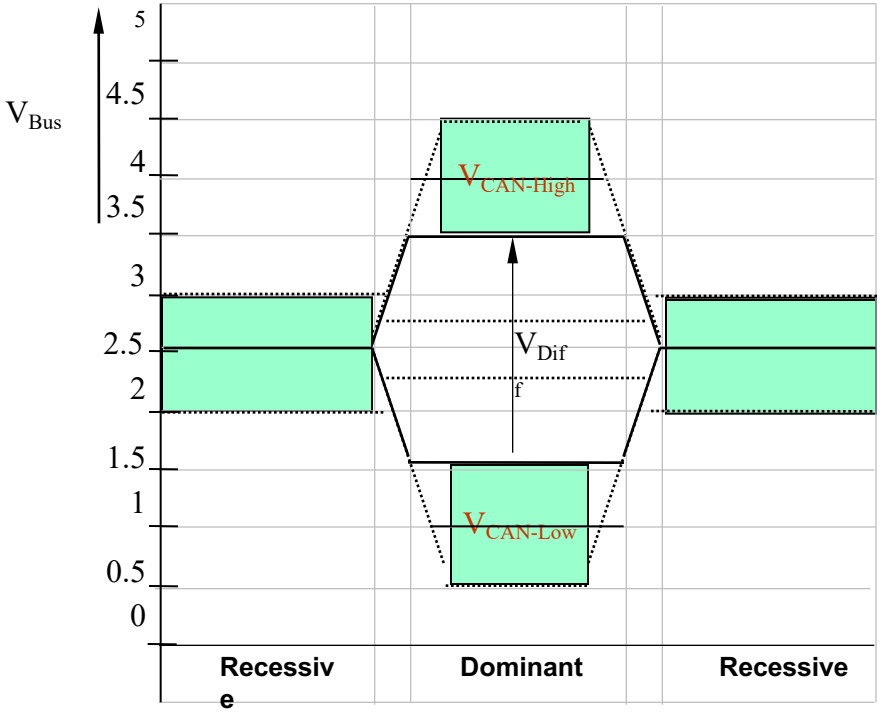
Physical Interface

- Three automotive standards: high speed differential (ISO 11898-2, SAE J2284), two-wire fault-tolerant (ISO 11898-3), and single-wire (SAE J2411)
- High Speed Interfaces (Class C)
  - Linear bus topology
  - Bus speed up to 1 Mbps
- Fault-tolerant interfaces (Class B)
  - Automatically switches to single wire reception when fault detected
  - Limited to 125kbps
  - Flexible topology: bus, star, ring
- Single Wire Interfaces (Class B)
  - Based on J1850-VPW technology with enhanced wakeup capability
  - Limited to 33.33kbps
  - Flexible topology: bus, star, ring

CAN voltage level

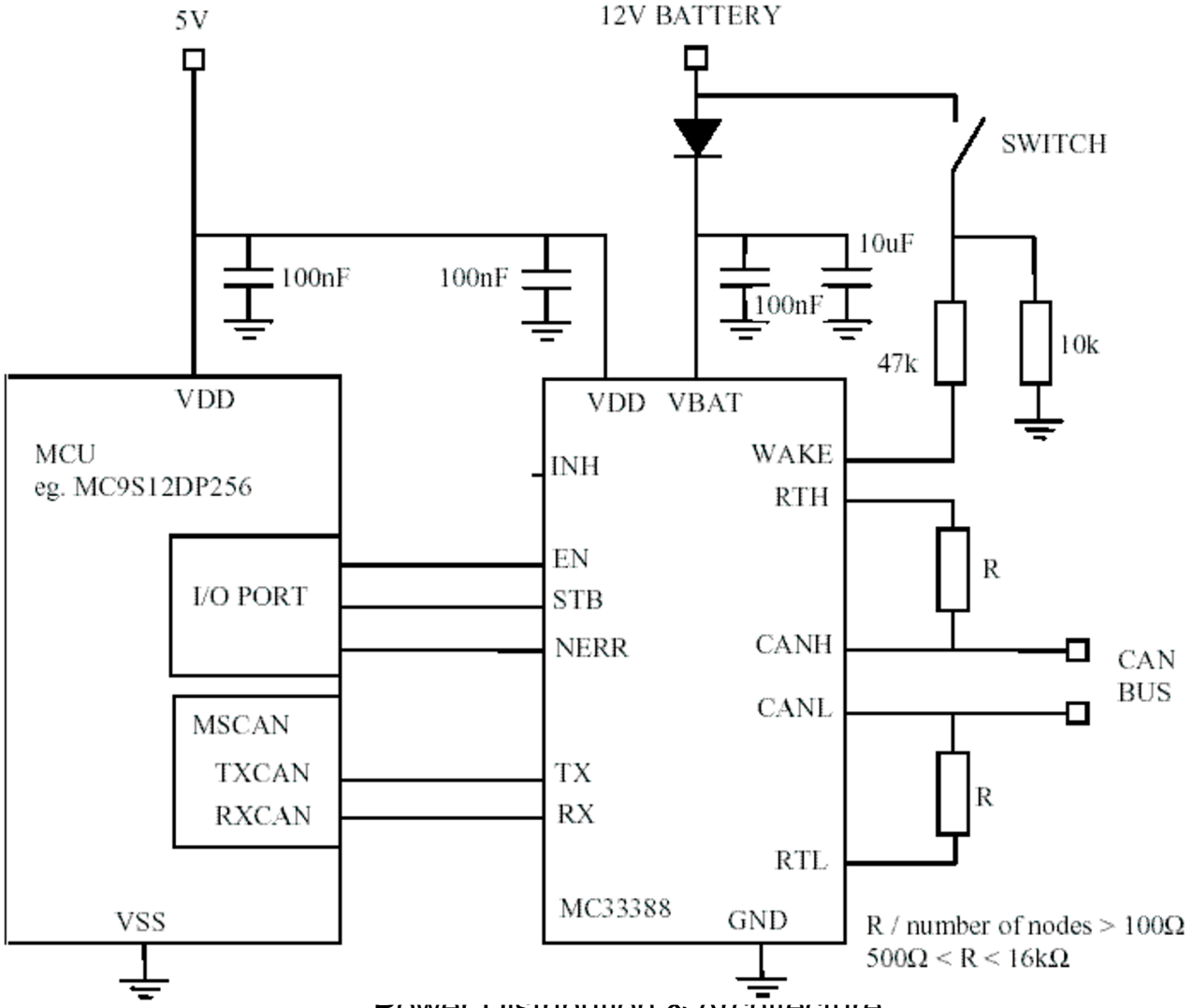


CAN CLASS -B



CAN CLASS -C

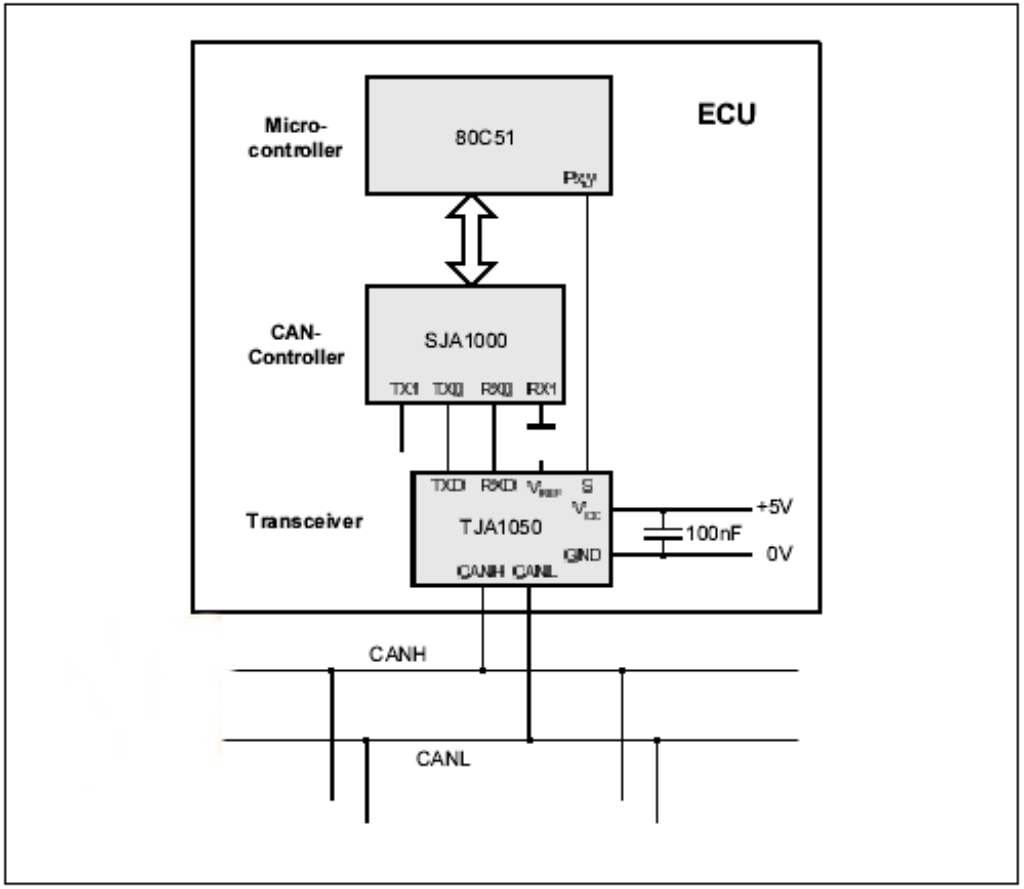
Typical CAN application circuit



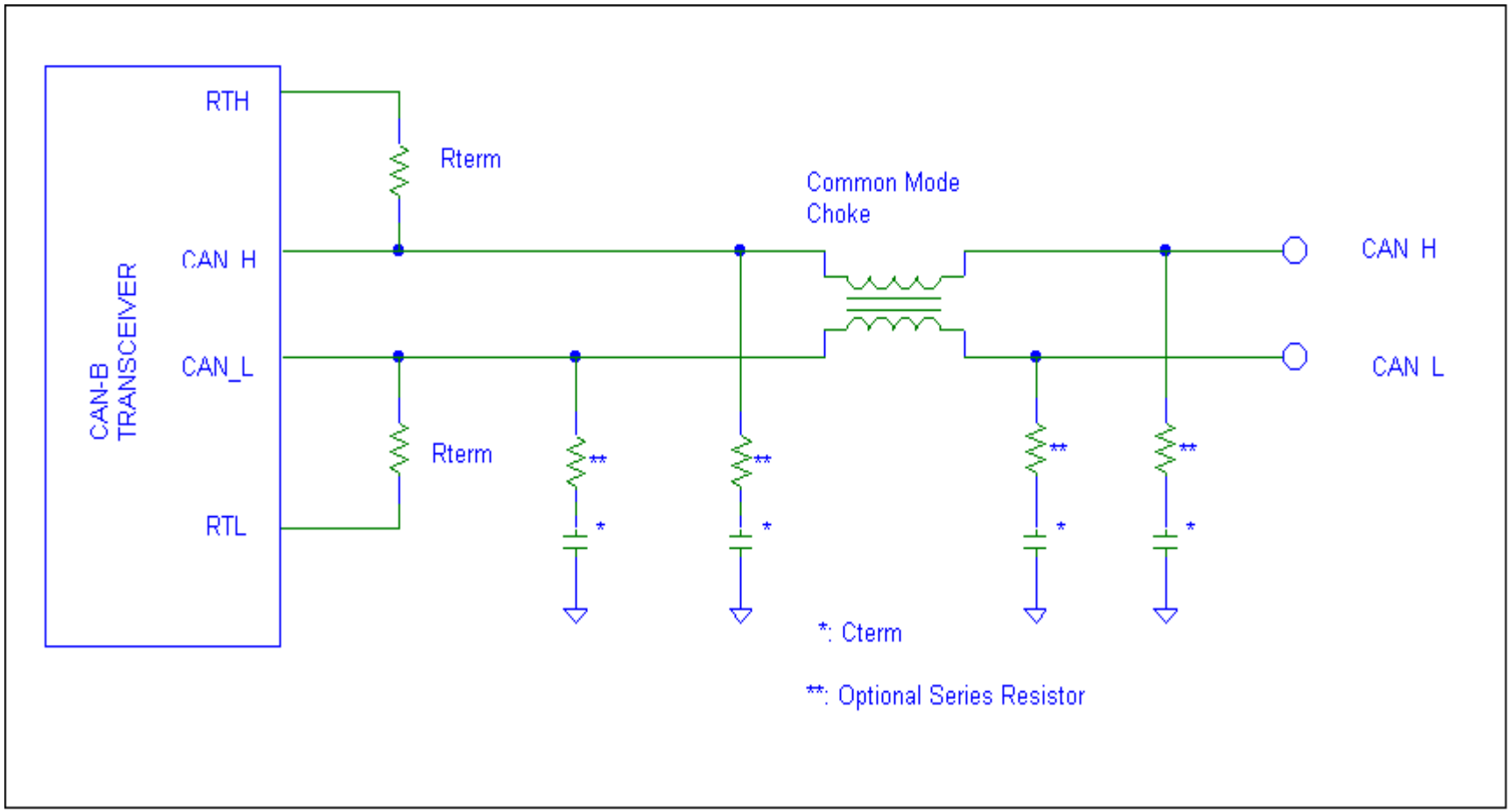
MC9S12DP256B : 16-Bit Microcontroller

- The MC9S12DP256B microcontroller unit (MCU) is a 16-bit device composed of standard on-chip peripherals :
  - **16-bit central processing unit (HCS12 CPU)**
  - **256Kb of Flash EEPROM, 4.0Kb of EEPROM**
  - **12.0K bytes of RAM**
  - **2 SCI**
  - **3 SPI**
  - **5 CAN 2.0 A, B software compatible modules (MSCAN12), and an Inter-IC Bus**

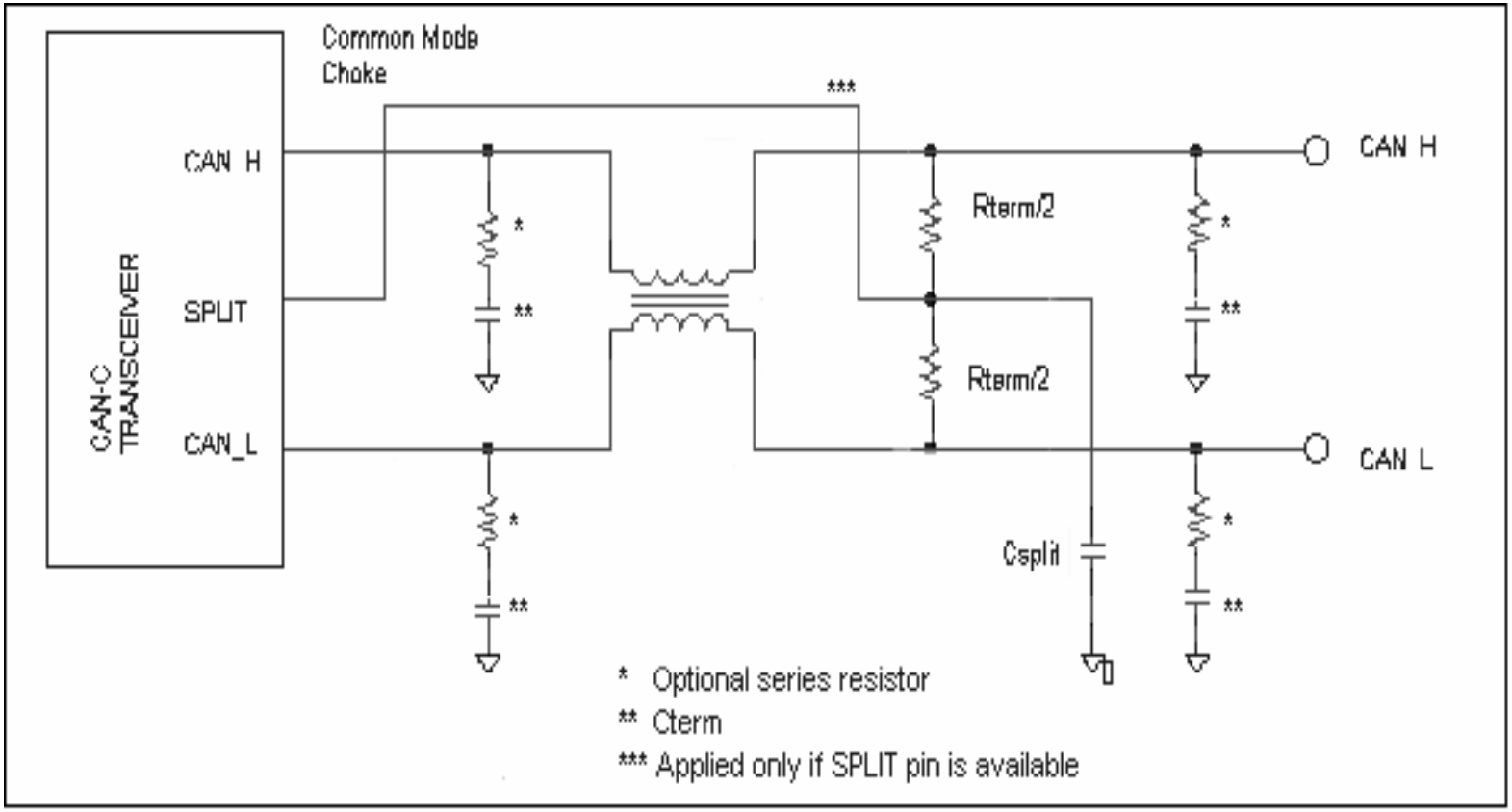
CAN ECU Configuration



CAN B Termination



CAN C Termination



Dominant Node Termination

CAN Dominant Nodes

- Body (83.33 & 125) Bus
  - CCN (Cabin Cluster Node)
  - TIPM (Totally Integrated Power Module)
- PT 500 Bus
  - Vehicle Specific
- DIAG 500 Bus
  - Double termination on TIPM

CAN bus speed at Chrysler

- 2004 -2006 Vehicles (PF-10647)
  - Body bus - CAN 83.33 (KBS) : Class B (FT)
  - Powertrain bus - CAN 500 (KBS) : Class C
  - Diagnostic bus - CAN 500 (KBS) : Class C
- Transition starting from 2007 vehicles (DC-10734)
  - Body bus - CAN 125 (KBS) : Class C
  - Powertrain bus - CAN 500 (KBS) : Class C
  - Diagnostic bus - CAN 500 (KBS) : Class C

A Gateway exchanges signals in various buses

CAN OSC Tolerance

Oscillator tolerance is determined by the requirement that bit values remain synchronized between recessive to dominant edges.

$$Tol = | f - fn | / fn \times 100\%$$

fn = nominal CAN clock freq, f = actual CAN clock freq.

Can oscillator tolerance at Chrysler

- PF 10647
  - CAN 83.33 KBS 0.30%
  - CAN 500 KBS 0.30%
- DC 10734
  - CAN 125 KBS 0.30%
  - CAN 500 KBS 0.15%

CAN Development Tools

- **CANalyzer:** Bus monitoring and analysis tool
- **CANoe**: Bus simulation, monitoring and analysis
- **CANstress:** Generate disturbances and analyze bus tolerance
- **CANscope:** Shows CAN waveforms identifying all message bits (PC based)
- Yokogawa CAN Oscilloscopes: Dedicated oscilloscopes for CAN
- **CANape:** Accesses and calibrates internal data contents within an ECU through CAN bus (uses CCP Driver)
- CAN I-Box: Interrogation, monitor and log signal values, send and monitor message.

CAN Analysis tools

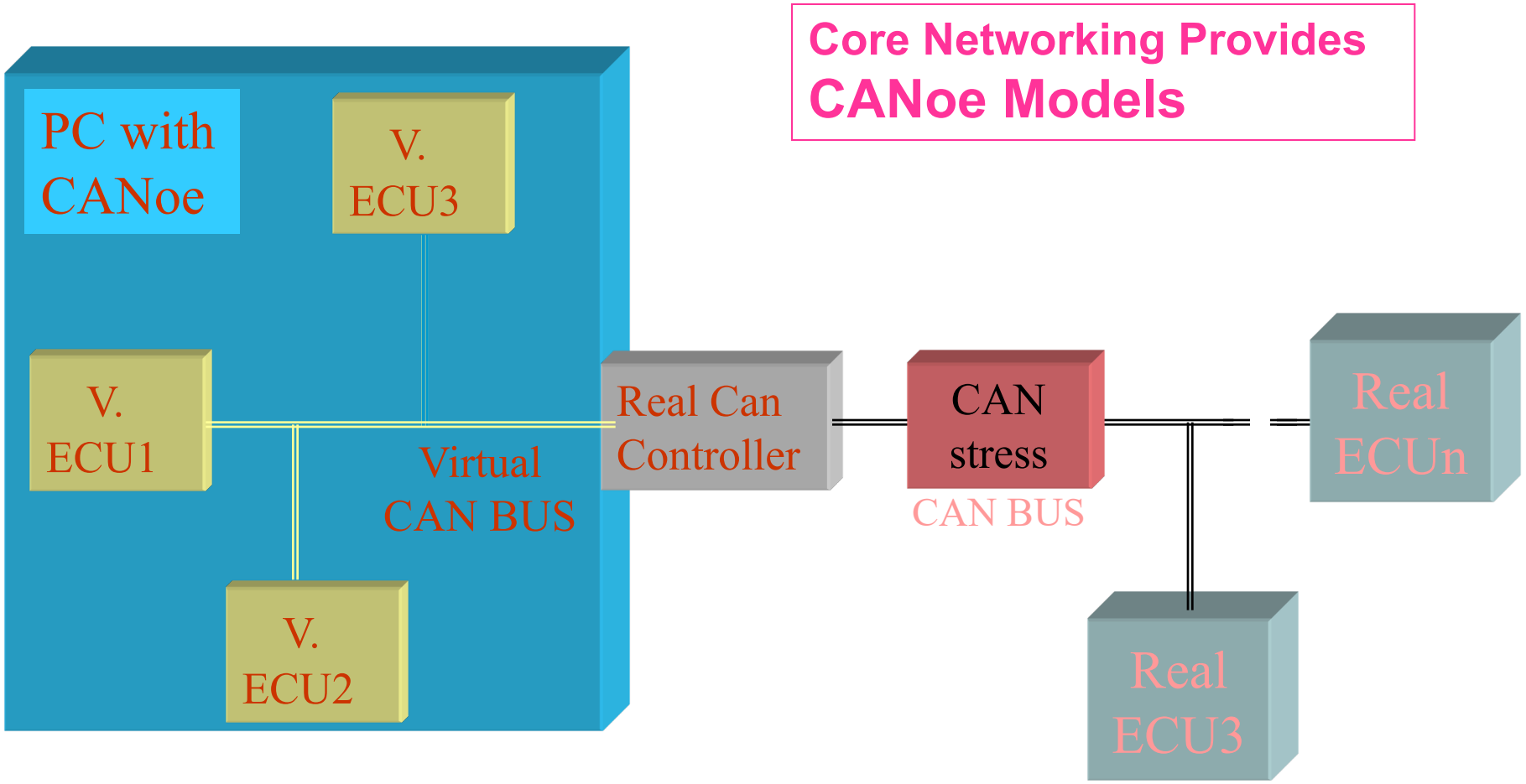
- CANalyzer and CANoe from Vector CANtech.
- S-CAT and Hercules from Dearborn Group Technology.
- NeoVI and Vehicle Spy from Intrepid Control Systems, Inc.

S-CAT/Hercules (Dearborn Group Inc.)

- Super CAN Analysis Tool (SCAT) software supports advanced communication with in-vehicle CAN networks (J2411 [dual wire: T module] and ISO 11898 [single-wire: S module]; low-speed, mid-speed, and high-speed).
- It is the ideal high-performance analysis tool for CAN network monitoring, simulation, diagnostics, and custom applications. Includes Scrolling, In-Place, and Signal monitors, as well as several views: Offline Buffer, Information, Diagnostics, Statistics.
- Hercules (GRYPHON) and SCAT hardware provide convenient Ethernet connection to the


Fresh Flower Arrangement
HTC
global services, inc.
www.htcinc.com
(248) 355-2500
GM
SCAT
Super CAN Analysis Tool
- WARNING -
Electrical Equipment Keep Away From All Liquids
DG

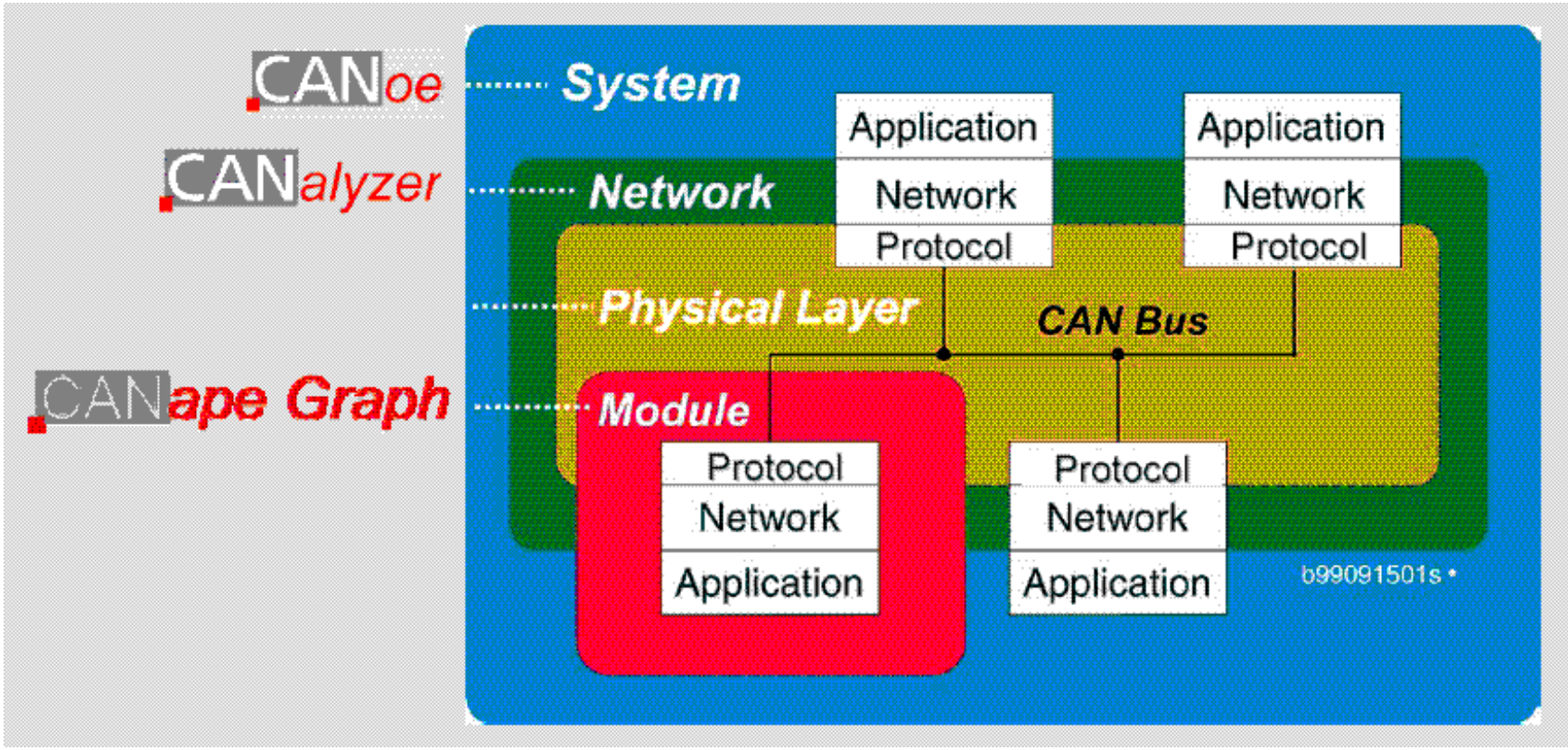
CANoe Simulation



CANoe (Vector-CANtech Inc.)

- CANoe (CAN Open Environment) is a powerful tool that supports the entire development process for networked systems, from planning through to placing the system in service.
- CANoe is a tool for all users involved in the project:
  - **Support is provided for the person responsible for the system in the form of function distribution, function testing, and integration of the entire system.**
  - **Simulation of remaining busses and environments provides suppliers with an ideal test environment**

Vector Tool Chain



Chrysler Networking Process

Specifications:

- **PF-10647**: ’04-’06 83.3k CAN B & 500k CAN C
- **DC-10734**: ‘07+, 125k Body CAN & 500k Powertrain & Chassis (PT&C) CAN
- **DC-10730**: 07MY+ CAN Comm. Driver spec.
- **LP- 384D-22**: ECU level test (’04-’06 program)
- **LP- 384D-23**: Vehicle level test (’04-’06 program)
- **LP-384D-27**: ECU level test (’07+ program)
- **LP- 384D-26**: Vehicle level test (’07+ program)

All specifications are available on ADRESS

CAN Driver Software

- Available from Vector CAN Tech
- For 2007MY suppliers may write their own driver compliant with DC-10730
- Driver source shall be transparent to Chrysler, i.e. no special treatment for those who chose to write their own drivers

ECU Level Test

- **LP- 384D-22 –** Formerly the “bread board” test.
- Must be performed by supplier prior to delivery of **any and all** software to Chrysler.
- Core Networking performs audits, may run the test again typically 12 weeks prior to major vehicle builds (e.g. S0 etc.).

Breadboard Simulation

- All supplier provided ECUs are connected to a real CAN bus
- CANoe simulations are used to analyze/monitor the bus activities
- Mutual communication, network sleep/ wakeup functions, bus on/off and hardware functions are verified

References

- *CAN* Controller *A*rea *N*etwork Technical Overview from Digital DNA from Motorola
- Motorola Application note AN2255/D Rev.0,2/2002 – MSCAN Low power applications
- http://www.vector-cantech.com/products/index.html
- S-CAT Hand-on Hercules 3.0 Training material from Dearborn Group
- CAN protocol Training material from Vector CANtech.
- Introduction to CAN training from Intrepid Control Systems, Inc.

References (contd…)

- Vector Application note – Introduction to Can from Vector CAN-tech
- Can: Network for thousands of applications outside automotive by Jack Shandle
- LIN Consortium – (www.lin-subbus.org)
- Motorola documentation – (http://mcu.motsps.com/documentation/index.html)
- Automotive Home Page – (http://www.mot-sps.com/automotive)
- Motorola LIN Software Inquiries – software.systems@helpline.sps.mot.com

LIN protocol

Dr. Subra Ganesan

Oakland University

LIN is developed for low cost networking

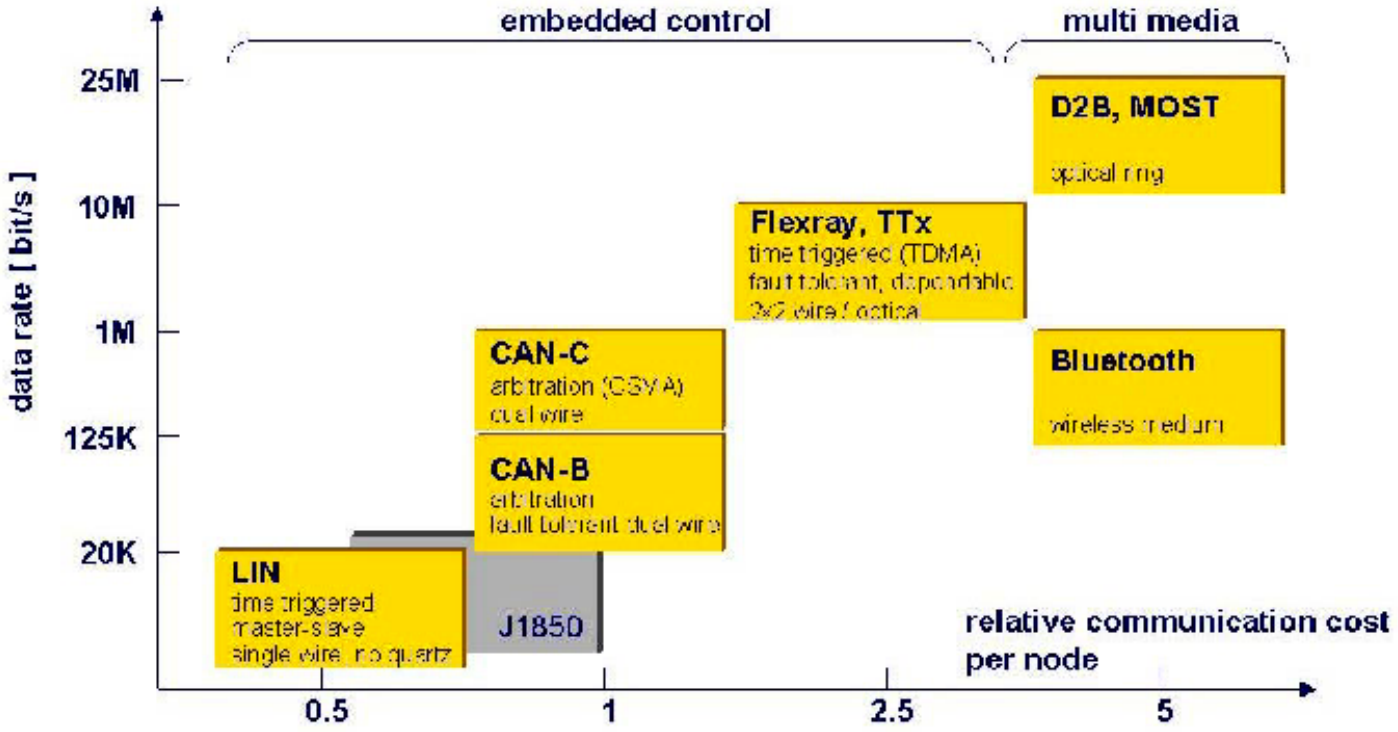


Figure 1: Major Network Protocols in Vehicles

Introduction

LIN- developed by LIN Consortium and a defacto standard. It is used to achieve cost-effective communication for intelligent *Sensors and Actuators* in a car.

LIN is used where the speed is less.

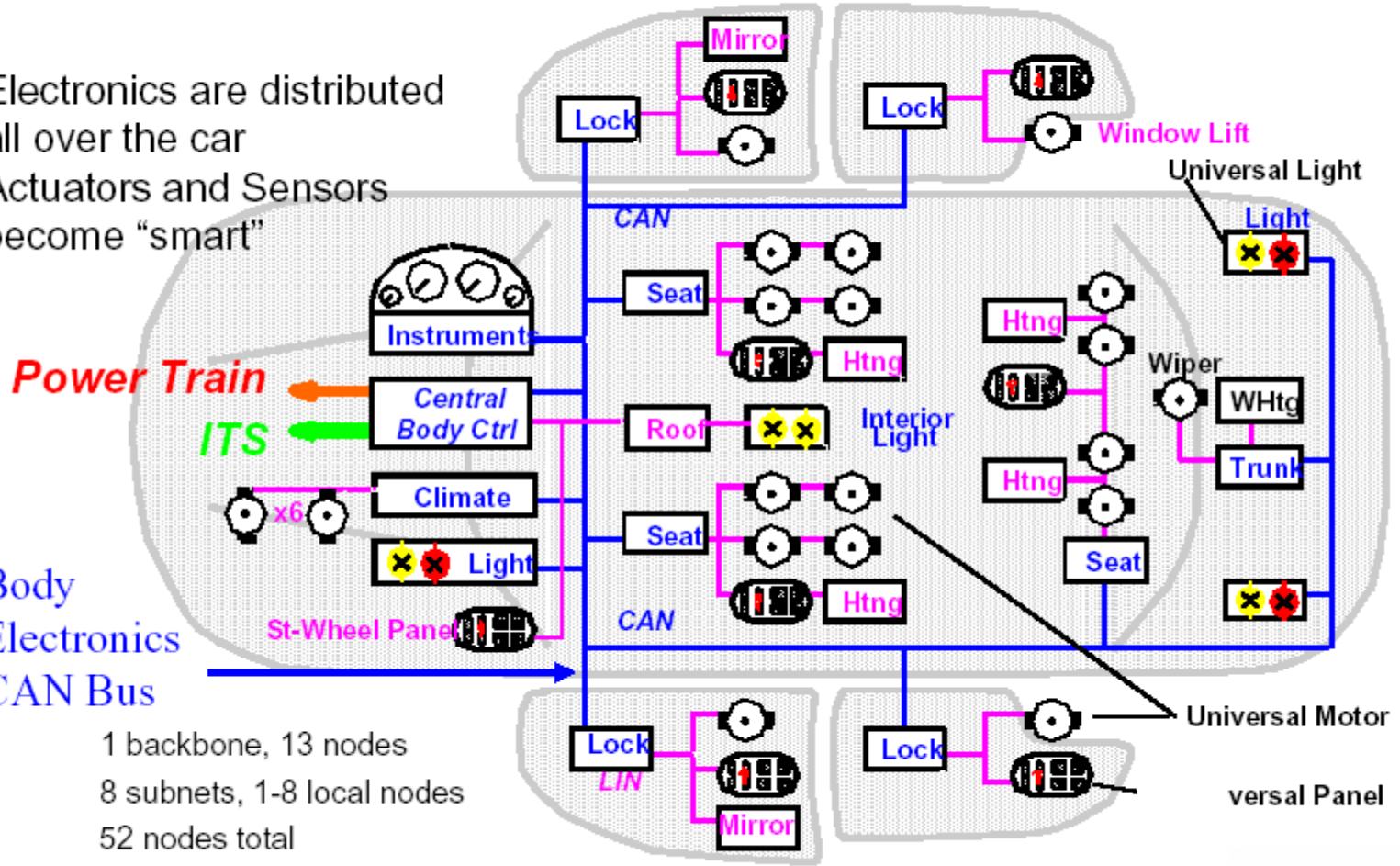
LIN network has: a Master and many Slaves.

No bus/ data collision. Schedule for transmission of data is fixed.

Detect Transmission Error by: Parity and Checksum.

Automotive Body Network

- Electronics are distributed all over the car
- Actuators and Sensors become "smart"

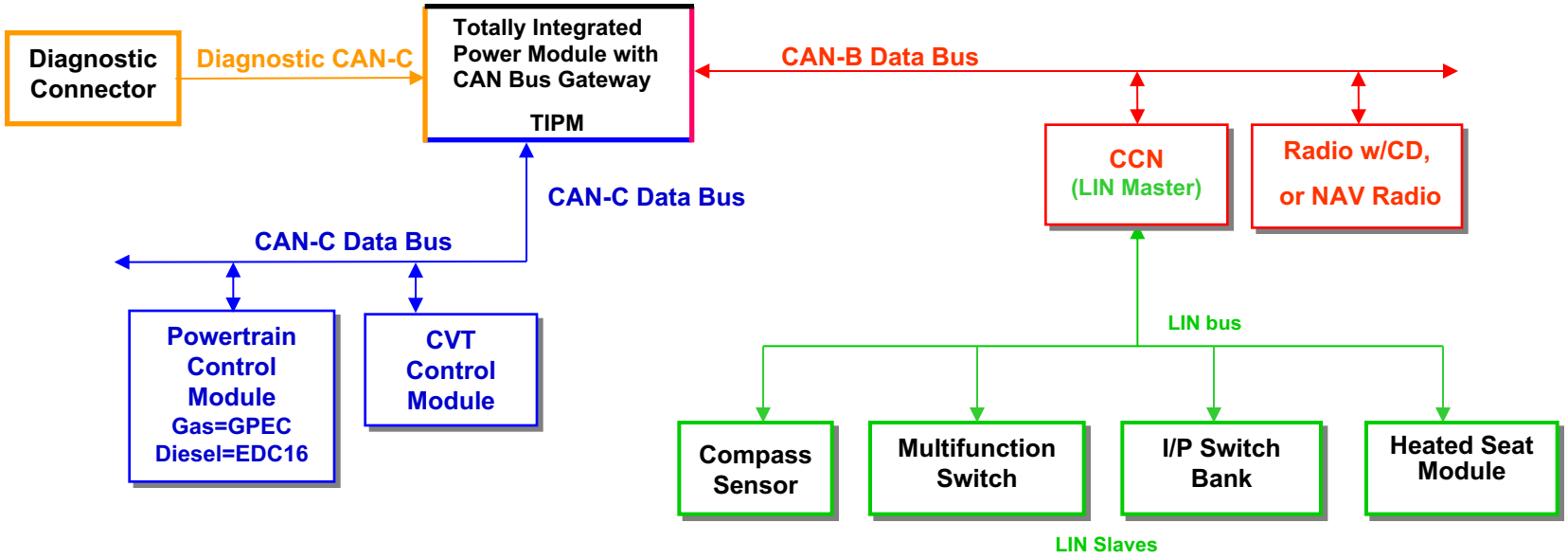


1 backbone, 13 nodes
8 subnets, 1-8 local nodes
52 nodes total

Benefits of LIN

- More Reliable Cars (Diagnostics)
- More Functionality at Lower Price
- Standardization of Interfaces and Components
- Faster Introduction of new Technologies
- Functional Extendibility

LIN System: Summary



LIN System: Summary

- Single master - Multiple slaves
- Master controls message scheduling - no arbitration - guaranteed latency times
- Selectable message length of 1-8 bytes
- Multicast reception with synchronization, without quartz or ceramics resonators in the slaves

LIN System: Summary (Contd.)

- Data checksum security
- Error detection
- Detection of defective nodes
- Single wire bus
- Variable baud rates -1 KBS to 20 KBS - (recommended 2.4, 9.6 10.4 & 19.2 KBS)
- Any micro with standard UART can be used

LIN Concept

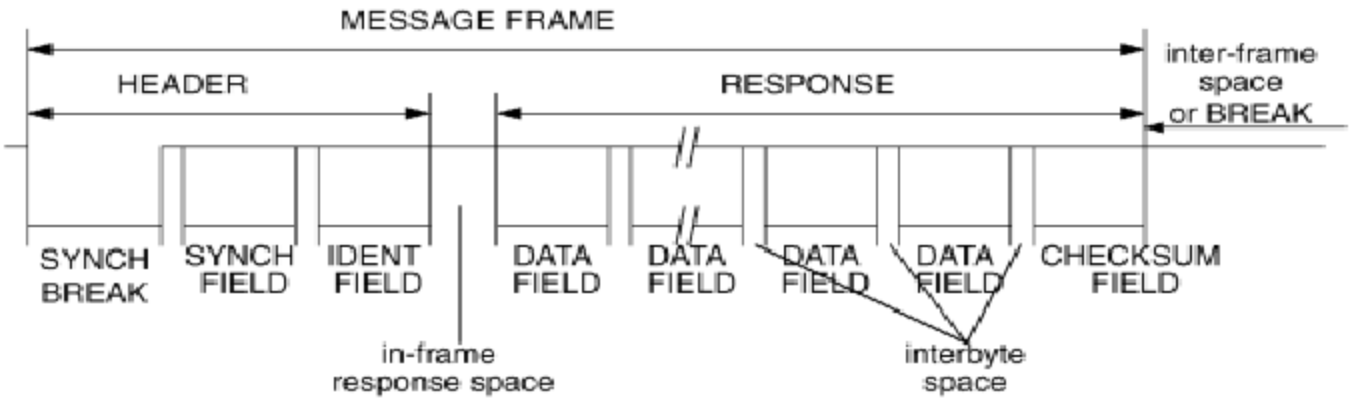
- Single-wire implementation (enhanced ISO 9141)
  → Low cost solutions (manufacturing and components)
- Speed up to 20Kbit/s (limited for EMI-reasons)
  → allows communication on hierachically distributed systems
- Single Master / Multiple Slave Concept
  → No arbitration necessary
- Low cost silicon implementation based on common UART/SCI interface hardware
  → Almost any Microcontroller has necessary hardware on chip!
- Self synchronization without crystal or ceramics resonator in the slave nodes
  → Significant cost reduction of hardware platform
- Guaranteed latency times for signal transmission
  → Predictable Systems

Master /Slave Protocol

- **Master Task**
  - Determines order and priority of messages.
  - Monitors Data and check byte and controls the error handler.
  - Serves as a reference with its clock base (stable clock necessary)
  - Receives Wake- Up Break from slave nodes
- **Slave Task**
  - Is one of 2-16 members on the bus
  - Receives or transmits data when an appropriate ID is sent by the master.
  - The node serving as a master can be slave, too!

Predictability

- Time Triggered Approach
  - Message Length is known
    - Number of transmitted data bytes is known
      → minimum length can be calculated
    - Each Message has length budget of 140% of it's minimum length
      → maximum allowed length is known
      → distance between beginning of two messages

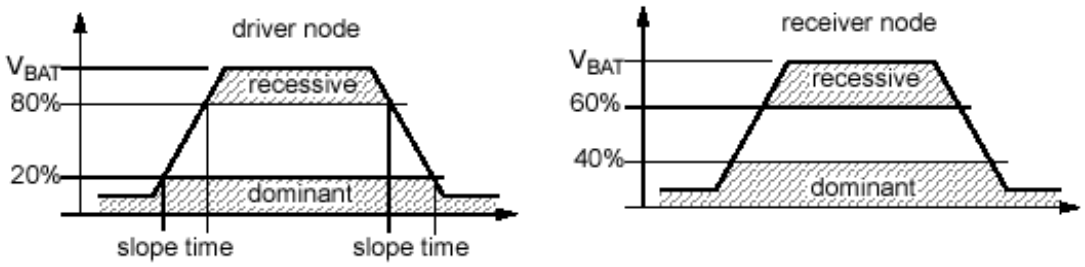


Physical layer

- Bus voltage

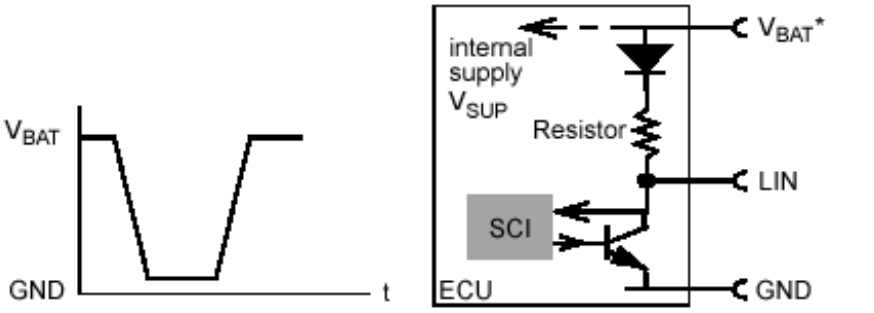
logical value	bit value	bus voltage
dominant	0	ground
recessive	1	battery

Logical and Physical Bus Values



Voltage Levels on the Bus Line

Physical layer (Contd.)



Concept of the Single-Wire Automotive Bus Interface

- Bus termination resistors

  Master - 1 kΩ

  Slave - 30 kΩ

- Maximum number of slaves = 15

Physical layer (Contd.)

- No message acknowledgement
- Master needs a quartz or ceramic resonator (±0.5%) , whereas slaves may have RC resonator with up to ±15% tolerance (with internal calibration)
- Supports sleep/wake up capability
- Either master or slave may wake up the bus

Message Frames

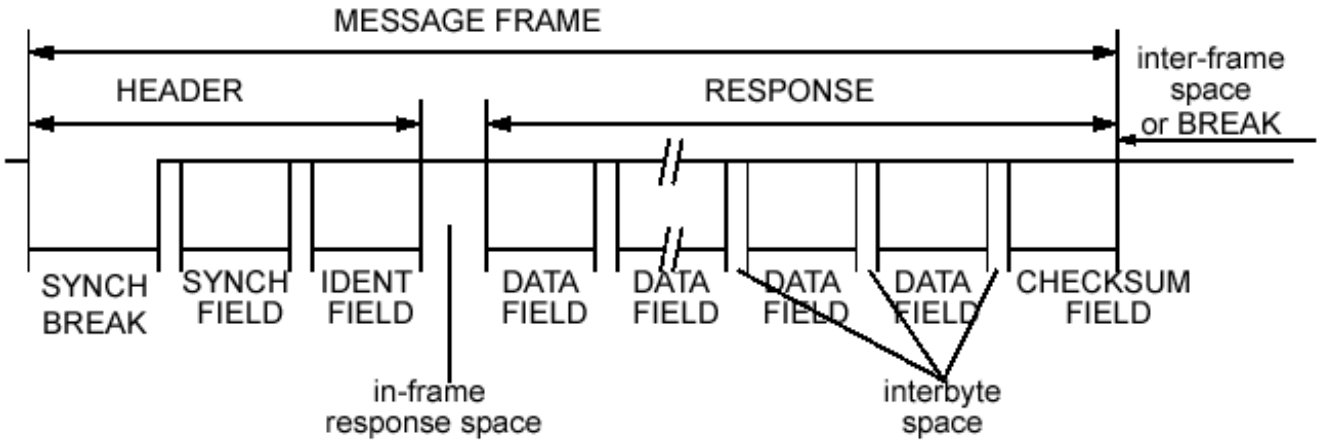
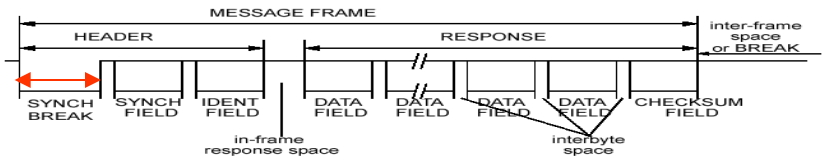


Figure 3.1: LIN MESSAGE FRAME



SYNCH BREAK FIELD

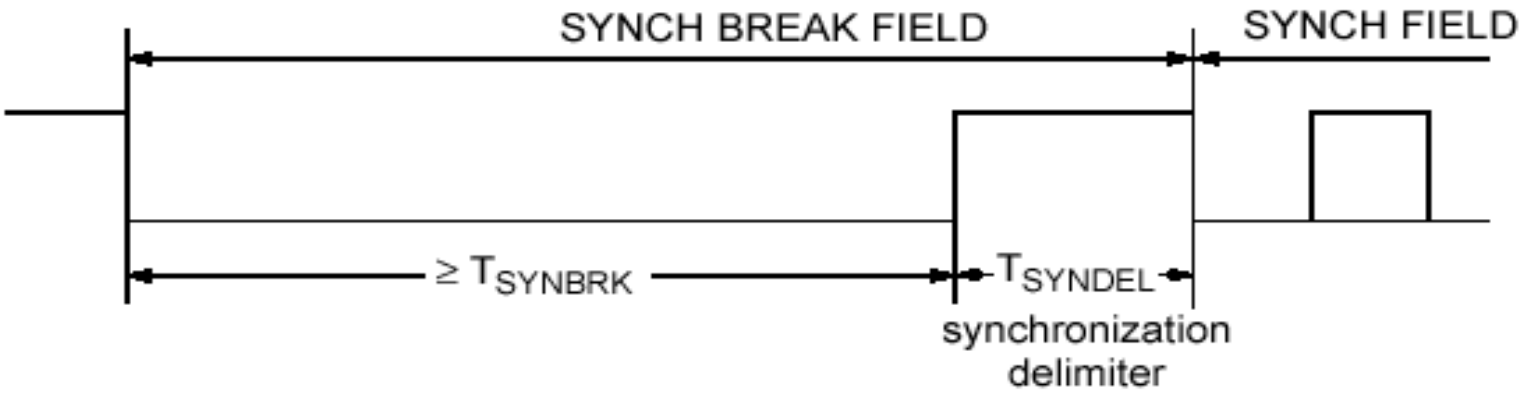
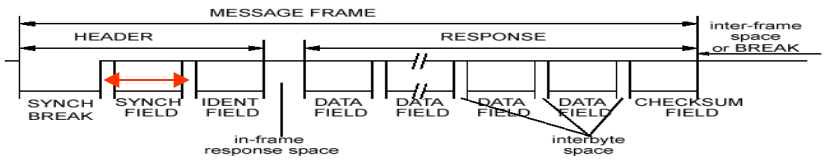


Figure 3.3: SYNCH BREAK FIELD

- Indicates start of a new message



SYNCH FIELD

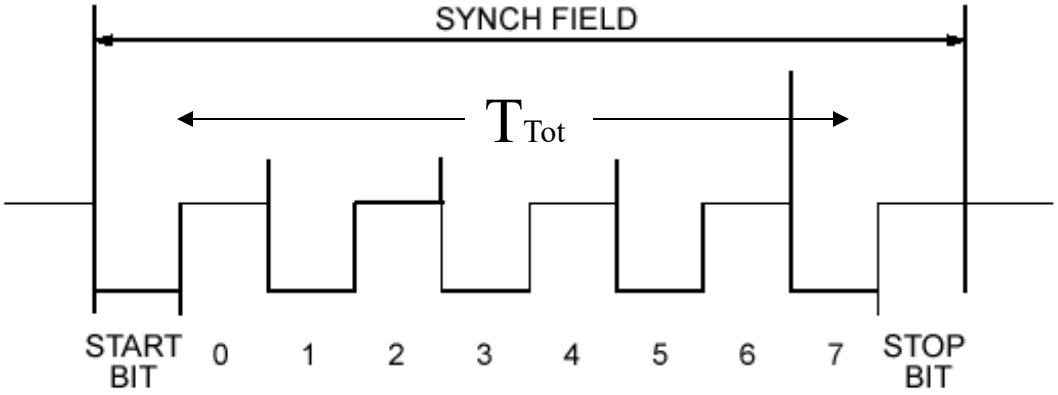
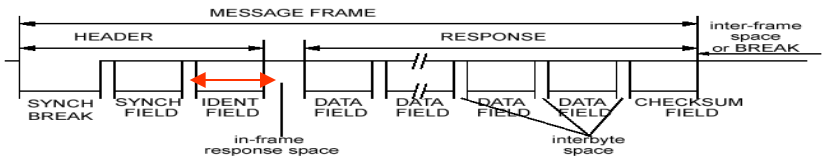


Figure 3.4: SYNCH FIELD

- Slave bit time is synchronized with the master
- $T_{bit} = T_{tot}/8$



IDENTIFIER FIELD

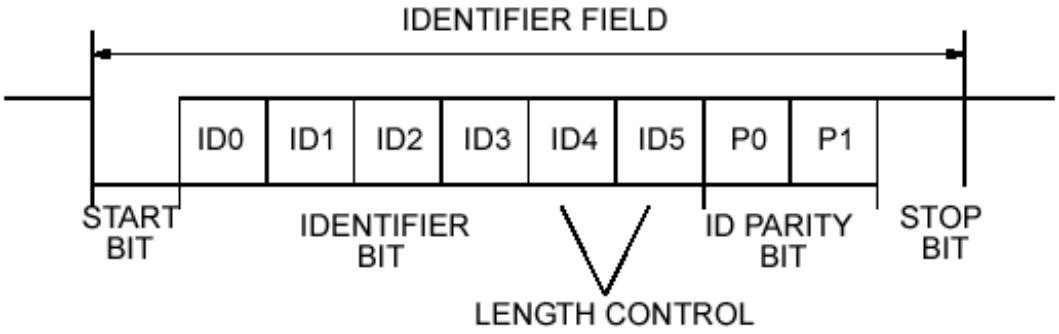
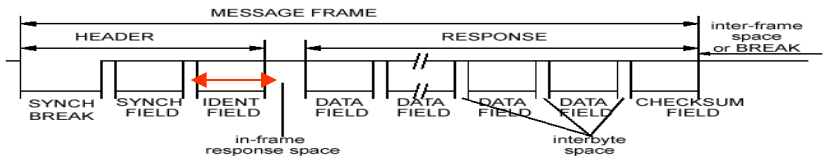


Figure 3.5: IDENTIFIER FIELD

- Bit 0-5 indicate 5-bit message ID (Identifier)
- Bit 4-5 may also indicate length
- Bit 6-7 are ID parity
- Bit 0-7 indicate Protected Identifier



ID FIELD

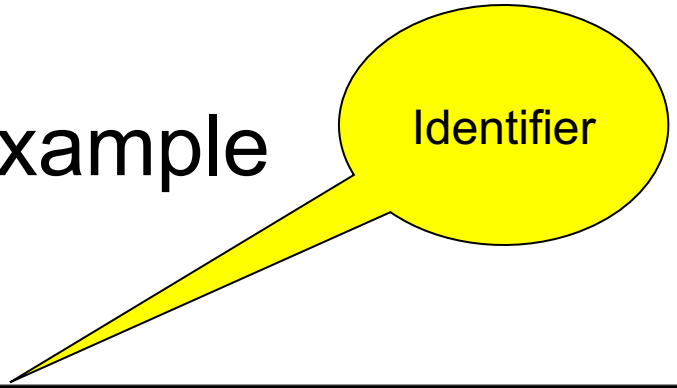
- 64 message identifiers are available (6-bit ID - 0h to 63h)
- 0h to 31h are 2-bit messages

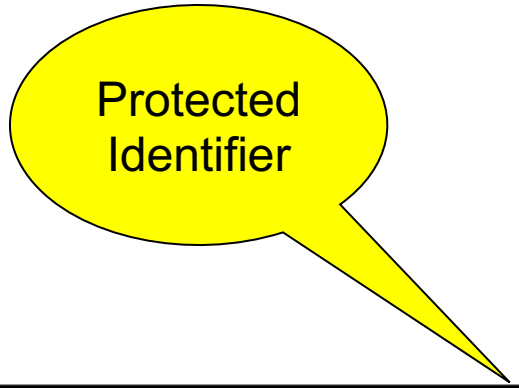
  32h to 47h are 4-bit messages

  48h to 63h are 8-bit messages
- Exceptions to the above are allowed

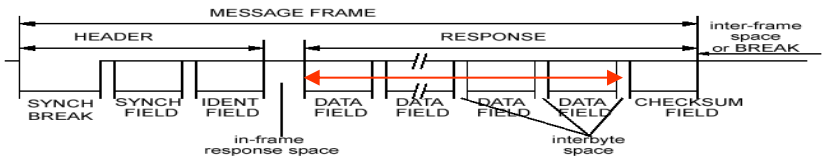
ID FIELD (Contd.)

- Example

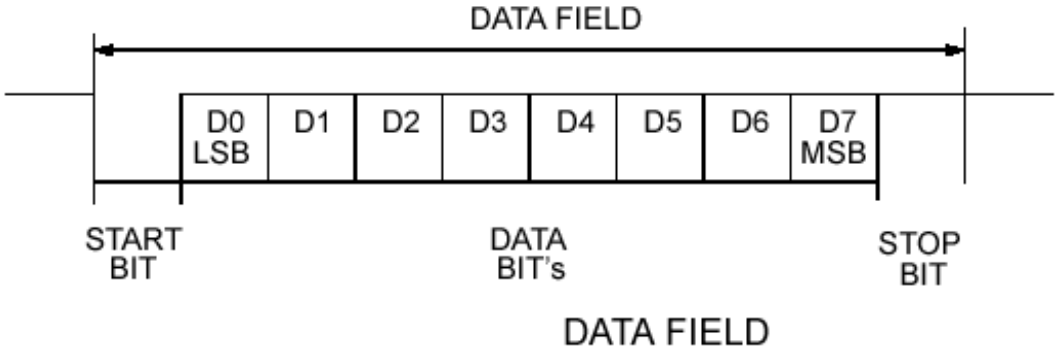




ID[0..5] Dec	ID[0..5] Hex	P0= ID0⊕ID1⊕ID2⊕ID4	P1= $\overline{ID1⊕ID3⊕ID4⊕ID5}$	ID-Field 7 6 5 4 3 2 1 0	ID-Field Dec	ID-Field Hex	# of Data Bytes
0	0x00	0	1	1 0 0 0 0 0 0 0	128	0x80	2
1	0x01	1	1	1 1 0 0 0 0 0 1	193	0xC1	2
2	0x02	1	0	0 1 0 0 0 0 1 0	66	0x42	2
3	0x03	0	0	0 0 0 0 0 0 1 1	3	0x03	2
4	0x04	1	1	1 1 0 0 0 1 0 0	196	0xC4	2
5	0x05	0	1	1 0 0 0 0 1 0 1	133	0x85	2



Response (Data Field)



- 1 - 8 data bytes
- LSB first

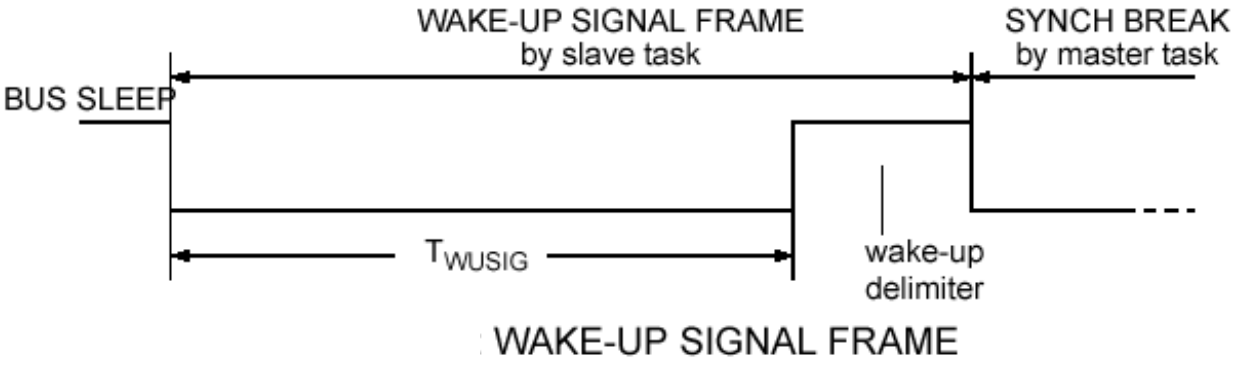
Reserved Identifier

Extended Frames

- ID field = 3Ch - Master Req (diagnosis Rq) - some values of first data byte are reserved
- ID field = 7Dh - Slave Resp (diagnosis Rsp)
- ID field = 3Eh - User defined
- ID field = 3Fh - For future expansion

Sleep/Wake up

- Sleep is requested by master with command frame (ID = 3Ch) with first byte = 0h
- Wake up frame

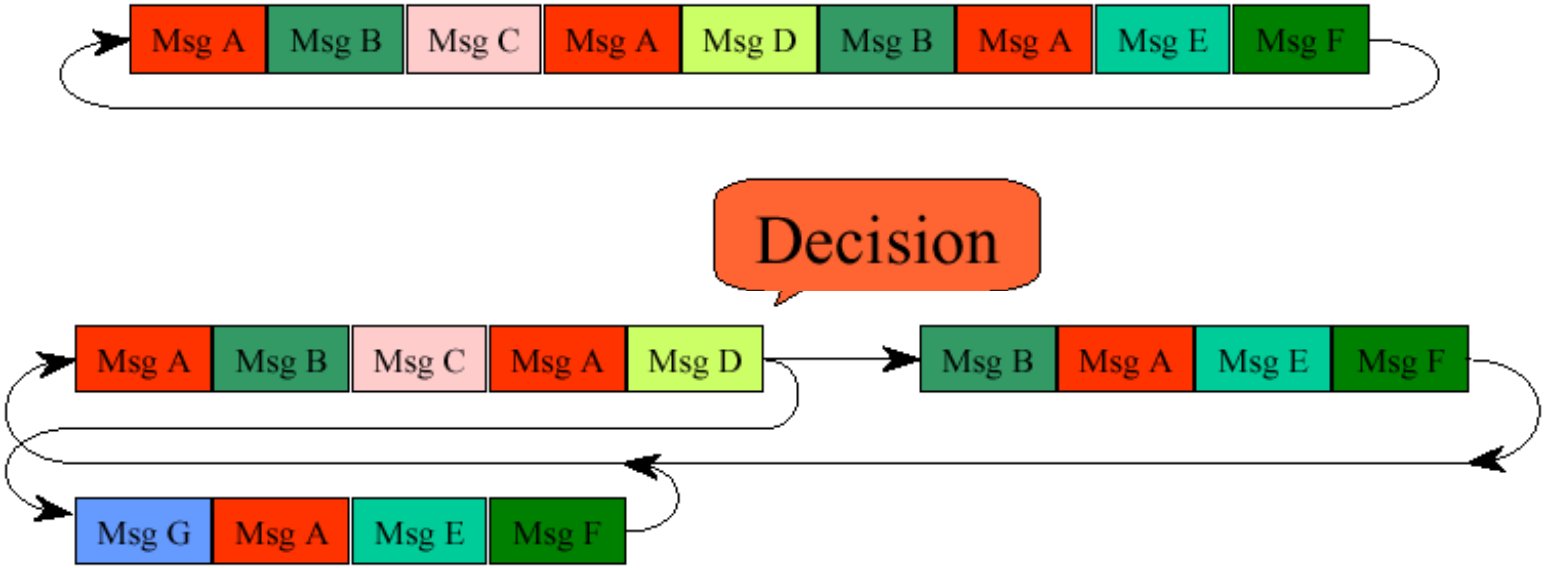


Error Handling

- Bit Error
- ID-Parity error
- Data checksum error
- Slave not responding error
- Inconsistent sync field error
- No bus activity
- Fault confinement depends on system requirement

Message scheduling

- Multiple schedule tables may be used



LIN Versions

- LIN 1.2/1.3
- LIN 2.0
  - Enhanced Checksum added
  - Slaves are auto baud rate
  - Slaves are configurable
  - ASICs can be used
  - Off-the-Shelf slaves will be available
  - Extensive diagnostics are added
- J2602
  - A subset of LIN 2.0
  - Bus speed is fixed at 10.4 kbps
  - Reduced diagnostics

A LIN 2.0 LIN master will support LIN 1.2/1.3, 2.0 and J2602 slaves

Chrysler LIN VMM
(Vehicle Message Matrix)

VMM describes the message contents with ID, message structure, and signal description

It is released in two formats

- EXCEL Format
  - Allows user-friendly browsing and printout
- LDF Format
  - Industry standard format for LIN messages that is used for code generation and driving a number of LIN tools



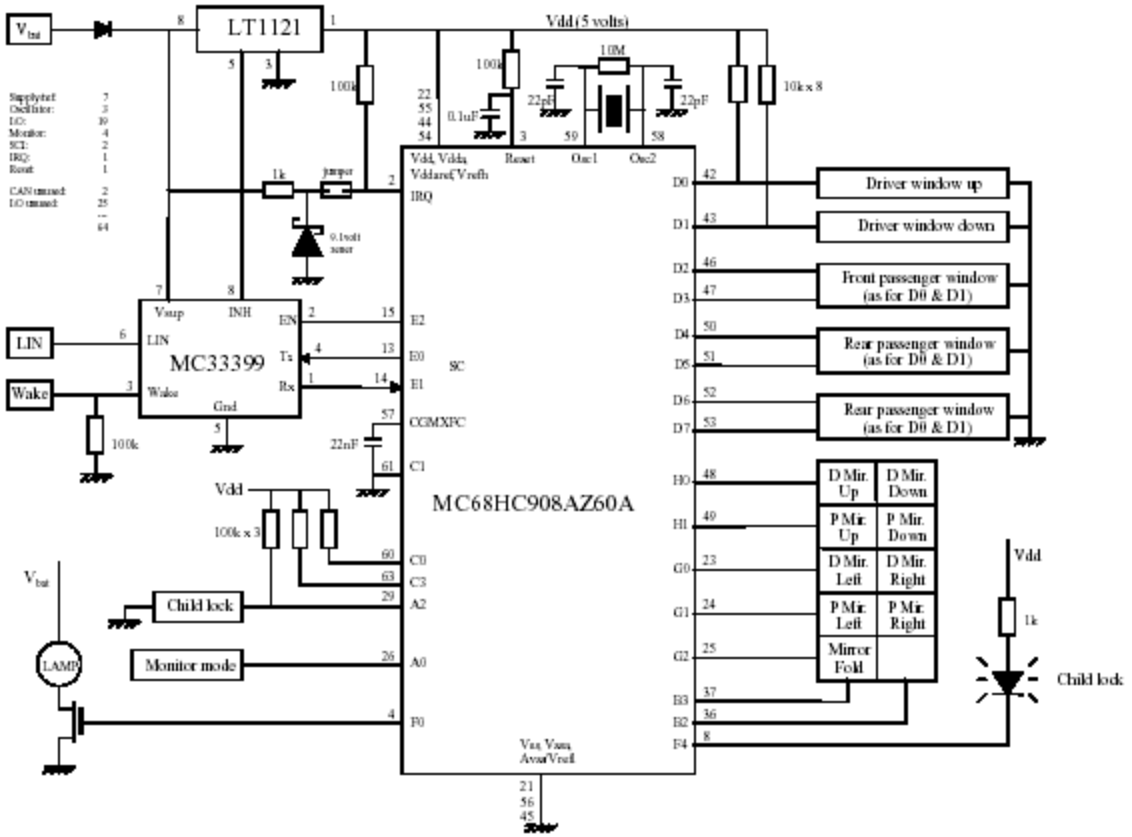


CAR door Keypad Using LIN

Traditional Door Keypads have micro switches that carry motor currents (> 1 Amps) to operate Windows/mirrors.

Use of LIN bus at each door, reduces the number of wires. The communication between the Keypad on the door and the Controller is through LIN bus.

Motorola, AN 2205/D: Keypad Module Circuit diagram



LIN Process at Chrysler

LIN Bus speed at Chrysler: 9.6 KBS

Backbone bus: CAN or J1850

Specifications:

- **PF-10752** : (Based on LIN 1.2) Existing programs
- **DC-10790**: (Based on LIN 2.0) '07RT +

(LIN 1.2 slaves will work with LIN 2.0 Masters)

Lab Procedures:

- **LP- 388D-25**: ECU level test
- **LP- 384D-24**: Network level test

All specifications are available on the ADRESS System

Monitoring & Emulation tools

- Volcano LINspector
- Vector CANoe-LIN

(Both driven by the LDF file)

ECU Level Test

- Use **LP- 388D-25**
- Must be performed by supplier prior to delivery of **any and all** LIN ECU to Chrysler.
- Core Networking performs audits, typically 12 weeks prior to major vehicle builds (e.g. S0 etc.).

Websites and Information

- LIN Consortium
  - (www.lin-subbus.org)
- Motorola documentation
  - (http://mcu.motsps.com/documentation/index.html)
- Automotive Home Page
  - (http://www.mot-sps.com/automotive)
- Motorola LIN Software Inquiries
  - software.systems@helpline.sps.mot.com

Breadboard Simulation

- All supplier provided ECUs are connected to a real CAN/LIN bus
- CANoe simulations are used to analyze/monitor the bus activities
- Mutual communication, network sleep/ wakeup functions and hardware functions are verified

Core Networking Contacts

- Supervisor: Kumar Singarajan (nick: ks30)
- CAN VMM: Fred Hartman (nick: fwh6)
- LIN VMM: Shahgir Ahmed (Nick: msa7)
- CAN mailbox access: Fred Hartman (nick: fwh6)
- CANoe simulation: Neelima Majjiga & Jon Sobocinski (nicks: nm548 & jgs27)
- CAN breadboard test: Neelima Majjiga & Jon Sobocinski (nicks: nm548 & jgs27)
- CAN & LIN hardware: Tim Potochick (nick: tp7)

References

- Personal discussions with employees from Harman/Becker
- Personal discussions with employees from Daimler Chrysler
- Industry Short course by Integrated Systems Solution on CAN
- Industry Short course by Integrated Systems Solution on LIN
- Shahgir Ahmed, Fred Hartman& Tim Potochick , "*Chrysler In-Vehicle Multiplexing with CAN/LIN Shahgir Ahmed, Fred Hartman& Tim Potochick*" power point presentation.